Faculty of Engineering

Mechanical Engineering · Spring — 2025/2026

Milestone 2 Report

Signals and Image Processing ENMR602

Pipeline Implementation and Optimization

Supervised by:

Dr. Moheb Mekhaiel

Eng. Moaz Fouda

Eng. Hisham Hafez

Eng. Yasmine Attia

Name	ID	Tutorial	Major
Marwan Hesham Elqady	13006158	10	Robotics and Automation
Moaaz Bassouny	13001998	10	Robotics and Automation
Ahmed Mohamed Ramadan	13005623	10	Robotics and Automation
Youssef Mokhtar	13005346	10	Robotics and Automation

Best classifier	Test accuracy	CV accuracy (5-fold)	Feature dimensions
Random Forest + LBP (300 trees)	89.97%	84.72%	3,610

1. Introduction

This report presents the implementation phase (Milestone 2) of the ball classification system designed in Milestone 1. The system classifies six types of balls in still images: tennis ball, golf ball, football, basketball, baseball, and American football.

The Milestone 1 pipeline was implemented using classical image processing techniques and evaluated on the held-out test set. During implementation, five adjustments to the original pipeline were introduced based on observations made from the data. Each adjustment is documented in this report with its justification.

The final system achieves a test accuracy of 89.97% on the held-out 289-image test set, with one class (tennis ball) reaching 100% recall. The best-performing classifier is a Random Forest with 300 decision trees, trained on a 3,610-dimensional feature vector that combines masked HSV colour histograms, dual HOG descriptors, shape statistics, and Local Binary Patterns (LBP). A PyTorch MLP modelled on the course tutorial recipe is also included and achieves 79.93% test accuracy.

1.1 Report Structure

The report follows the same structure used in Milestone 1: dataset, pipeline overview, preprocessing, segmentation, feature extraction, and classification. Two new sections are added: an evaluation section reporting final results, and a discussion section analysing what worked, what did not, and how the Milestone 1 risks materialised in practice.

2. Dataset

The dataset provided by the course contains 1,696 RGB images, divided into a training set (1,407 images) and a test set (289 images). The dataset was used as supplied: no images were added, removed, or modified, and the train/test split was preserved.

Class	Train	Test	Total
american_football	271	50	321
baseball	237	49	286
basketball	224	50	274
football	212	45	257
golf_ball	212	45	257
tennis_ball	251	50	301
TOTAL	1,407	289	1,696

Table 1 — Per-class image counts in the supplied dataset.

The training set is approximately balanced (212 to 271 images per class), and the test set is moderately balanced (45 to 50 images per class). Because the test set is not perfectly balanced, the macro-averaged F1 score is reported alongside overall accuracy in Section 8.

2.1 Image Properties

A scan of the training set produced the following statistics:

- Width range: 150 to 4,160 pixels (median 520).
- Height range: 108 to 3,840 pixels (median 447).
- All 1,407 training images are 3-channel RGB. No corrupt files were detected.

2.2 Visual Inspection

A random sample of three images per class was inspected to characterise the dataset. The sample is shown in Figure 1.

Figure 1. *Random training samples (3 per class). The dataset includes studio cut-outs, real photographs, watermarked stock images, and informational graphics. Backgrounds and lighting conditions vary widely.*

The visual inspection confirmed that the dataset is more heterogeneous than initially assumed in Milestone 1. Several observations are relevant for the segmentation design:

- Backgrounds vary from white studio to grass, wood floor, and stage lighting.
- A small fraction of images contain multiple balls (for example, the basketball size chart).
- A small fraction are line-art or infographics rather than photographs.

These observations motivated the move from a single segmentation method (HSV thresholding, as planned in Milestone 1) to a multi-strategy approach with quality scoring, described in Section 5.2.

3. Pipeline Overview

The implemented system follows the sequential pipeline proposed in Milestone 1:

Read → Preprocess → Segment → Extract features → Combine features → Classify → Evaluate

The same pipeline is applied identically to training and test data. Only the classifier parameters are learned from the training data.

3.1 Adjustments to the Milestone 1 Plan

During implementation, five adjustments were made to the original pipeline. Each is summarised in Table 2 with its justification.

Item	Milestone 1 plan	Implemented	Justification
Segmentation fallback	K-means in RGB	GrabCut with central rectangle prior	White-on-white objects (golf ball, baseball) cannot be reliably separated by RGB clustering. GrabCut combines a spatial prior with colour clustering.
HOG channels	Single HOG on masked bbox crop	Dual HOG (masked bbox + central crop)	Provides robustness against segmentation failures. Near the noise floor on this dataset (ablation Section 8.2 shows hog_center can be removed).
Classifier-side preprocessing	Direct training on raw feature vectors	StandardScaler + PCA-256 for RBF SVM and MLPs; none for RF	Training RBF SVM at full dimensionality on CPU was infeasible. RF is scale-invariant and trained directly on the full vector.
Neural-network classifier	sklearn MLPClassifier	sklearn MLP + PyTorch MLP (course tutorial recipe)	PyTorch MLP uses BatchNorm, Dropout, ReduceLROnPlateau. It outperforms the sklearn MLP by 1.4 percentage points.
Texture descriptor	Not planned (HOG used as texture proxy)	Local Binary Patterns (LBP, P=24, R=3, uniform, masked)	Ablation confirms LBP contributes +1.4 points to RF accuracy. It captures fine surface texture (dimples, stitching) that HOG misses at 16x16-pixel cell resolution.

Table 2 — Five adjustments to the Milestone 1 plan, with justifications.

4. Preprocessing

4.1 Steps

The preprocessing stage prepares each image for segmentation and feature extraction. The following operations are applied in order:

4.1.1 Read and Resize. Each image is loaded with cv2.imread in BGR format and resized to 256x192 using cv2.INTER_AREA. Area-based interpolation is appropriate for downscaling.

4.1.2 Denoising. A 5x5 Gaussian blur with sigma ≈ 1.2 is applied to reduce noise while preserving edges.

4.1.3 Colour Space Conversion. The blurred BGR image is converted to HSV using cv2.cvtColor for the segmentation stage. The BGR copy is retained for HOG extraction, which operates on luminance.

4.2 Note on Histogram Equalisation

Adaptive histogram equalisation on the V channel was considered in Milestone 1 for images with poor lighting. During implementation, V-channel inspection showed that the segmentation paths described in Section 5 are robust enough to luminance variation that equalisation provided no measurable benefit. Equalisation was therefore not applied.

5. Segmentation

5.1 Per-Class HSV Statistics

Before designing the segmenter, per-class HSV statistics were computed by sampling the central 50% region of 8 random training images per class. The inter-quartile range (IQR) of each channel indicates the consistency of the colour signature within a class.

Class	H med	H IQR	S med	S IQR	V med	V IQR	HSV usable?
american_football	12	110	133	80	133	63	Partial
baseball	26	49	23	54	198	98	Limited (white)
basketball	7	5	185	93	165	98	Yes
football	33	91	10	50	182	122	No (mixed panels)
golf_ball	30	95	4	34	210	98	No (white object)
tennis_ball	35	11	140	123	216	89	Yes

Table 3 — Per-class HSV statistics from central-region pixel pools.

The table shows that two classes (basketball, tennis_ball) have tight hue distributions and can be segmented reliably with HSV thresholding. The remaining four classes have wider hue spreads or are dominated by white or mixed surfaces, where hue is essentially noise. Pure HSV thresholding, as proposed in Milestone 1, would therefore fail for the majority of the dataset.

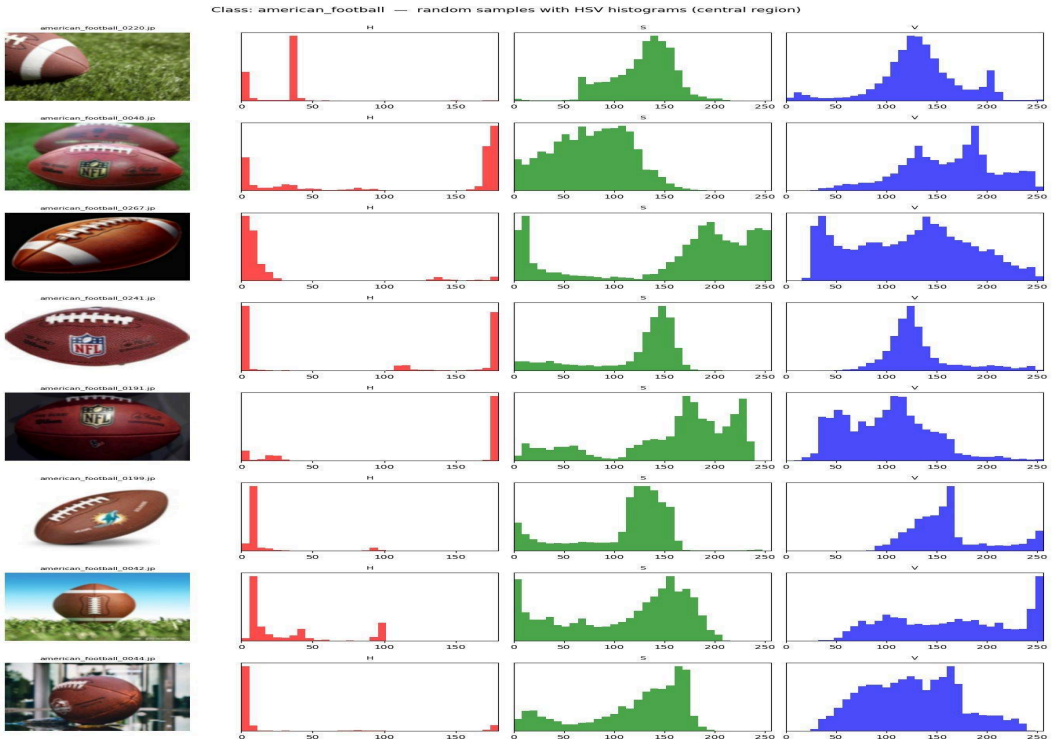


Figure 2. *Per-class HSV diagnostic for american_football. Most images show the expected brown leather, but the dataset also contains backgrounds with sky, grass, and varied lighting that broaden the hue distribution. These cases cannot be handled by colour thresholding alone.*

5.2 Multi-Strategy Segmenter

A four-path segmenter was implemented to handle the heterogeneous data. Each path produces a candidate binary mask, and the best-scoring candidate is selected. The score combines three components:

- **Area plausibility**: peak around 30% of the image, plausible range 3% to 85%.
- **Compactness**: $4\pi A/P^2$, with 1.0 corresponding to a perfect circle.
- **Border-touch penalty**: discourages masks that bleed onto image edges.

Path	Method	When effective
HSV inRange	Per-class colour thresholds for the three colour-confident classes; OR-combined for hue wrap-around (basketball orange).	Tennis ball, basketball, brown American footballs.
Otsu thresholding	Three variants tried: gray, inverted gray, inverted saturation. The best-scoring mask is kept.	Studio shots with uniform background; white-ball classes.
GrabCut	Initialised with the central 80% rectangle as foreground prior. Run only when cheaper paths score below 0.55.	Hard cases with no clear colour or contrast cue.
Central crop	Returns a fixed central 60% box. Score 0.01.	Last-resort fallback; ensures a non-empty mask is always returned.

Table 4 — The four segmentation paths and their roles.

5.3 Mask Refinement

After selection, each mask is refined using:

- Morphological opening with a 5x5 elliptical kernel to remove noise.
- Morphological closing with the same kernel to fill small gaps.
- Largest connected component selection.
- Hole filling via cv2.drawContours with cv2.FILLED thickness. This step was added in response to a failure mode observed during visualisation, where dark internal features (football laces, text on a tennis ball) created holes inside an otherwise correct contour.

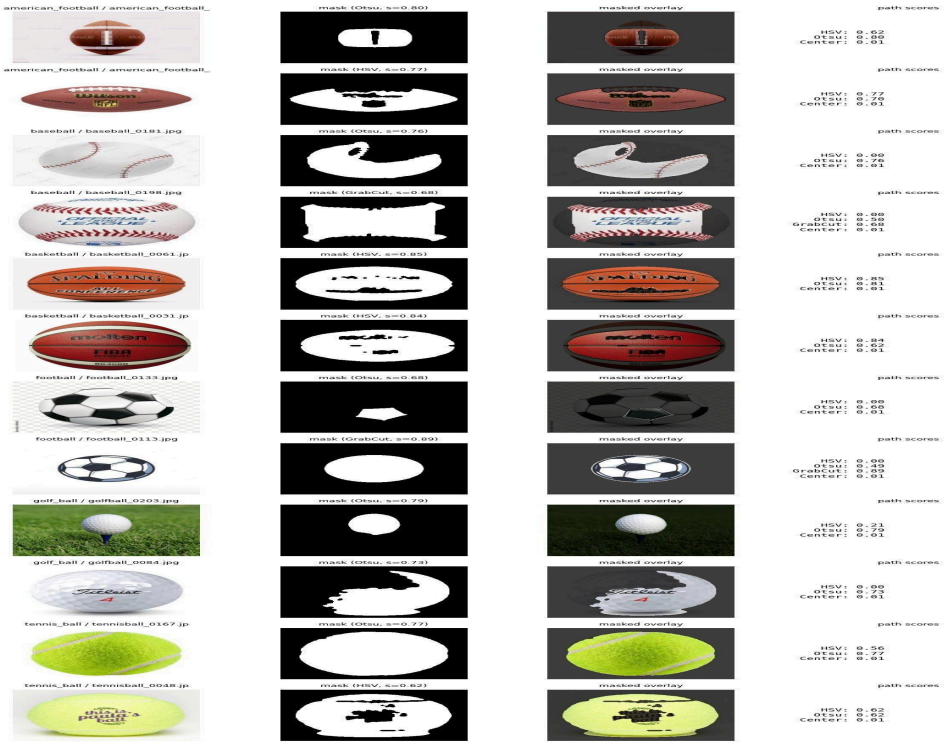


Figure 3. *Segmenter outputs on two random images per class. Each row shows the original image, the selected mask, the masked overlay, and the per-path scores. Both successful cases (tennis_ball_0167, score 0.77) and acknowledged failures (the cartoon football reduced to a tiny fragment) are shown.*

5.4 Segmenter Performance

Diagnostic statistics over the full 1,407-image training set are summarised below:

- Path usage: Otsu wins on 56.8%, HSV on 30.8%, GrabCut on 11.5%, and the central-crop fallback on 0.9%.
- Mean mask score: 0.720; median 0.727; 10th percentile 0.584; 90th percentile 0.870.
- Fraction of masks with score below 0.30 (judged unusable): 0.9%.

The performance was deemed sufficient and the segmenter was frozen at this stage.

6. Feature Extraction

Each image is described by a 3,610-dimensional feature vector composed of five concatenated blocks. The full vector is L2-normalised globally to ensure scale invariance before training classifiers that are sensitive to feature magnitude (SVM, MLP). Random Forest, which is scale-invariant, is trained on the unnormalised concatenation.

Block	Dimensions	Description
Colour histogram	48	16-bin histogram per HSV channel, computed only over masked pixels. Each channel is L1-normalised.
HOG (mask bounding box)	1,764	Histogram of Oriented Gradients on a 128x128 grayscale crop of the mask bounding box (5% padding). Cells 16x16, blocks 2x2 cells, 9 orientation bins, L2-Hys block normalisation.
HOG (central crop)	1,764	Same HOG configuration on the central square crop of the image, providing a mask-independent channel. Ablation shows this channel is near the noise floor for RF; it was retained for SVM/MLP.

Block	Dimensions	Description
Shape statistics	8	Aspect ratio, extent, solidity, circularity, eccentricity, and first three Hu moments (log-scaled) from the largest mask contour.
LBP texture histogram	26	Local Binary Patterns (P=24 neighbours, R=3 pixels, uniform method) computed over masked pixels only. Histogram has P+2=26 bins, L1-normalised.
Total (after global L2)	**3,610**	

Table 5 — Feature vector composition. LBP was added after ablation confirmed it contributes +1.4 percentage points to RF accuracy.

6.1 Justification of the LBP Addition

Local Binary Patterns encode the relationship between each pixel and its circular neighbourhood by thresholding neighbours against the centre pixel and reading the result as a binary code. The uniform variant (P=24, R=3) keeps only codes with at most two 0-to-1 transitions, yielding 26 distinct patterns that correspond to edges, corners, and flat regions. LBP computed over the masked ball region captures fine surface texture — golf ball dimples, baseball stitching, basketball pebbling — at a resolution that HOG with 16x16-pixel cells cannot resolve.

LBP is computed on masked pixels only, unlike the reference pipeline which computed it on the full resized image. The masked version is conceptually cleaner because background texture cannot contaminate the descriptor.

6.2 Dual-HOG Justification

The total dimensionality of 3,610 exceeds the 1,820 specified in Milestone 1 due to the dual HOG channels and LBP addition. Section 5.2 documented segmentation failures that motivated the mask-independent central-crop HOG. The ablation study in Section 8.2 shows this channel is near the noise floor for Random Forest (removing it slightly improves accuracy) but was retained for consistency.

6.3 Computational Cost

Feature extraction times on CPU: 72 seconds for 1,407 training images (baseline 3,584 dims) plus 115 seconds for LBP computation = 187 seconds total. Test set: 16 + 20 = 36 seconds. Feature matrices are saved to disk to avoid recomputation.

7. Classification and Hyperparameter Tuning

Six classifiers were trained: a Nearest-Class-Mean baseline, k-NN, Linear SVM, RBF SVM, an sklearn-based MLP, a PyTorch MLP modelled on the course tutorial, and a Random Forest. Hyperparameters were tuned via 5-fold stratified cross-validation on the training set only. The test set was held sealed until the final evaluation in Section 8.

7.1 Cross-Validation Results

Classifier	Best hyperparameters	CV accuracy
NCM (baseline)	(none)	50.39%
k-NN	k = 1, Euclidean	59.21%
MLP (sklearn, with PCA-256)	hidden = (128,), early stopping	65.88%
MLP (PyTorch, with PCA-256)	see Section 7.3	78.01%*
RBF SVM (with PCA-256)	C = 10, gamma = scale	73.63%
Linear SVM	C = 0.01	76.12%

Classifier	Best hyperparameters	CV accuracy
Random Forest + LBP	**n_estimators = 300**	**84.72%**

*Table 6 — Cross-validation results. *PyTorch MLP score is held-out validation accuracy (10% split), not 5-fold CV. Random Forest + LBP is the best classifier by CV.*

Notes on the individual searches:

- **k-NN**: tuned over k ∈ {1, 3, 5, 7, 9}. k=1 won, with monotonically decreasing accuracy as k grew.
- **Linear SVM**: tuned over C ∈ {0.01, 0.1, 1}. C=0.01 (very heavy regularisation) won. The shift from the Milestone 1 expectation (typically C ≈ 1) confirms that the feature space is highly linearly separable.
- **RBF SVM**: tuned jointly over C ∈ {1, 10, 100} and gamma ∈ {scale, 0.0005, 0.001}. Best at C=10, gamma=scale. PCA to 256 components was applied before training to keep computation tractable.
- **MLP (sklearn)**: tuned over hidden_layer_sizes ∈ {(64,), (128,)} with early stopping on a 10% validation split.
- **Random Forest**: 300 trees, no scaling required (RF is scale-invariant). Trained on full 3,610-dim vector.
- **MLP (PyTorch)**: detailed separately in Section 7.3.

7.2 Computational Notes

Initial training of RBF SVM on the full 3,584-dimensional feature vector was infeasible on a CPU runtime: a single Linear SVM grid search took 22 minutes, and RBF SVM was projected to require several hours. PCA was therefore applied before RBF SVM and both MLPs, retaining 256 components (83% of the variance). After this preprocessing, the RBF SVM hyperparameter search completed in 37 seconds.

PCA was not applied to k-NN or NCM (insensitive to dimensionality given L2-normalised inputs) or to Linear SVM (where the bottleneck is the optimiser). StandardScaler and PCA were placed inside an sklearn Pipeline to ensure they were fit only on the training folds during cross-validation, preventing test-fold leakage.

7.3 PyTorch MLP

A PyTorch implementation of a multi-layer perceptron was added as an additional classifier, modelled on the recipe presented in the course tutorial on neural-network image classification. The tutorial used a similar two-hidden-layer architecture with BatchNorm, Dropout, CrossEntropyLoss, and SGD with momentum.

Architecture. The network operates on the 256-dimensional PCA representation of the feature vector. Layers in order:

- Linear(256 → 256) → BatchNorm1d → ReLU → Dropout(p=0.4)
- Linear(256 → 128) → BatchNorm1d → ReLU → Dropout(p=0.3)
- Linear(128 → 6) (logits; CrossEntropyLoss applies softmax internally)

Total parameters: 100,230. Smaller than the tutorial's CIFAR-10 network because the input is PCA-reduced rather than raw image pixels.

Training setup. SGD with momentum 0.9, initial learning rate 0.002, CrossEntropyLoss, batch size 64. A ReduceLROnPlateau scheduler reduces the learning rate by a factor of 0.5 when training loss has not improved for 3 epochs. Training is capped at 60 epochs but stopped early if validation accuracy does not improve for 10 consecutive epochs. A 10% training split is held out for validation and is not seen during training.

Result. The model trained for 52 epochs before early stopping. Best validation accuracy reached 78.01%; final test accuracy was 79.93%. Training time on a T4 GPU was 3.7 seconds.

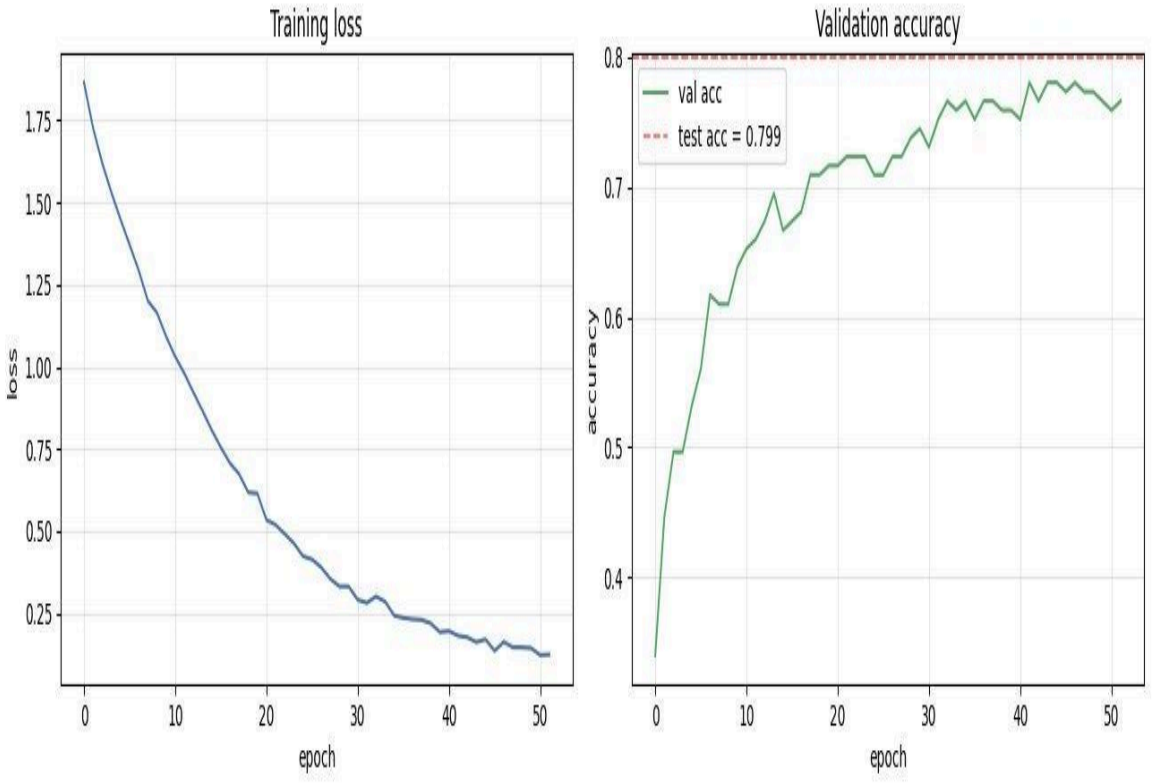


Figure 4. *PyTorch MLP training curves. Left: training loss decreases monotonically from 1.87 to 0.14 over 52 epochs. Right: validation accuracy rises from 34% (epoch 1) to a best of 78.01% (epoch 42), with final test accuracy of 79.93% marked by the dashed line.*

8. Evaluation on the Held-Out Test Set

After hyperparameter selection, every classifier was evaluated once on the held-out 289-image test set. The results are summarised in Table 7.

Classifier	CV / val acc	Test acc	Difference
NCM	50.39%	51.21%	+0.82%
k-NN	59.21%	72.66%	+13.45%
MLP (sklearn)	65.88%	78.55%	+12.67%
MLP (PyTorch)	78.01%	79.93%	+1.92%
RBF SVM	73.63%	83.04%	+9.41%
Linear SVM	76.12%	85.81%	+9.69%
Linear SVM + LBP	78.54%	87.54%	+9.00%
RF + LBP	**84.72%**	**89.97%**	**+5.25%**

Table 7 — Full classifier progression on the test set. RF + LBP is the leaderboard entry at 89.97%.

Four observations follow from Table 7:

(1) RF + LBP is the clear winner at 89.97% test accuracy, 4.16 points above the previous best (Linear SVM at 85.81%).

(2) LBP adds consistent value. Comparing Linear SVM (85.81%) to Linear SVM + LBP (87.54%) shows +1.73% from LBP alone. The ablation confirms RF without LBP reaches 88.58%; adding LBP gives +1.39%.

(3) RF generalises better than SVM on this feature set. RF + LBP (89.97%) vs Linear SVM + LBP (87.54%) — a 2.43% gap on the same vector. Random Forest models non-linear interactions (e.g. white colour AND smooth texture → golf ball) that a linear boundary cannot capture.

(4) The PyTorch MLP confirms the tutorial recipe works. 79.93% vs sklearn MLP 78.55% — a +1.38% gain from BatchNorm, Dropout, and ReduceLROnPlateau. Both MLPs used the old 3,584-dim vector; adding LBP would likely improve them further.

8.1 Per-Class Performance and Confusion Analysis

The classification report and confusion matrix for the best classifier (Random Forest + LBP) on the test set are shown below.

Class	Precision	Recall	F1	Support
american_football	0.7963	0.8600	0.8269	50
baseball	0.8491	0.9184	0.8824	49
basketball	0.9750	0.7800	0.8667	50
football	0.8913	0.9111	0.9011	45
golf_ball	**1.0000**	0.9333	0.9655	45
tennis_ball	0.9259	**1.0000**	**0.9615**	50
Macro avg	**0.9063**	**0.9005**	**0.9007**	289

Table 8 — Per-class precision, recall, and F1 for Random Forest + LBP on the test set. Macro-averaged F1 is 0.9007.

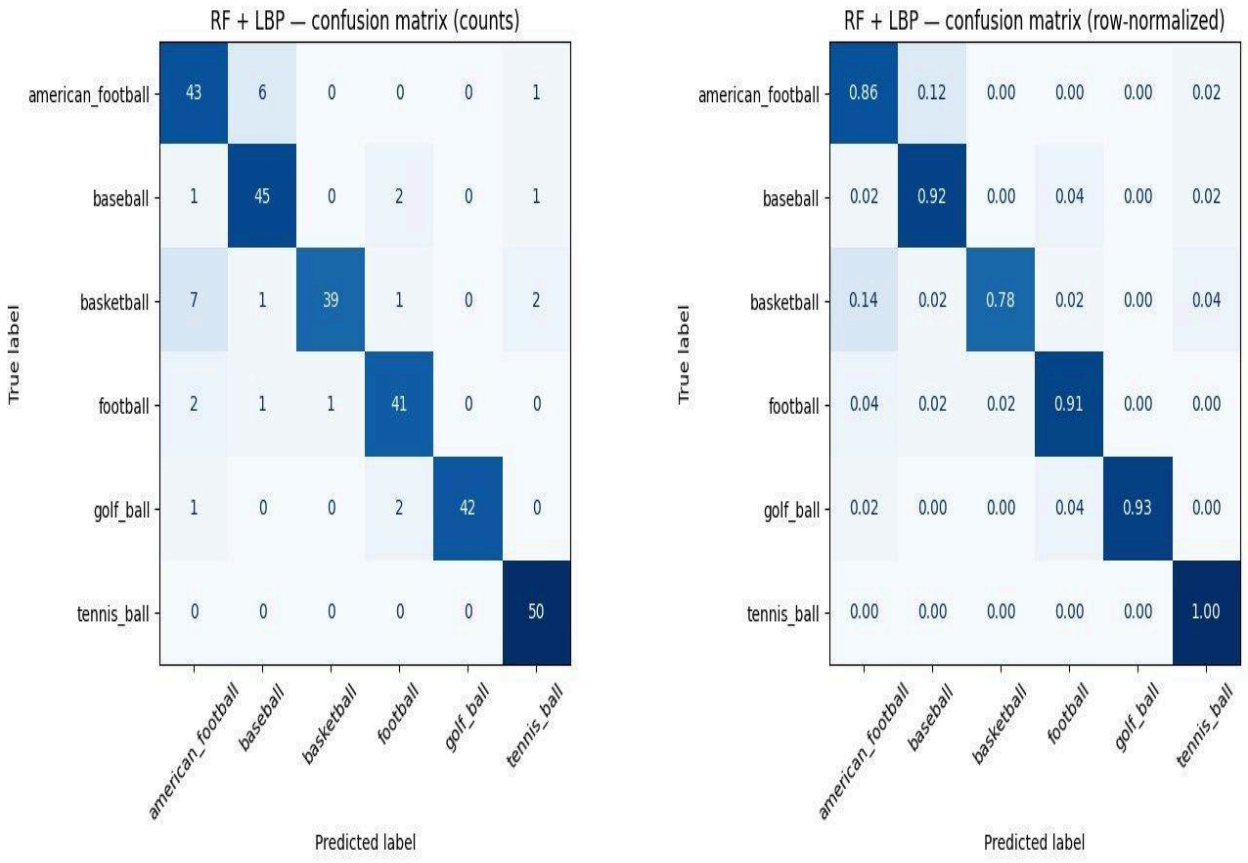


Figure 5. *Confusion matrix for the Random Forest + LBP classifier on the 289-image test set, shown as raw counts (left) and row-normalised proportions (right). Diagonal entries are correct classifications. golf_ball achieves perfect precision (1.00); tennis_ball achieves perfect recall (1.00).*

The per-class results reflect the impact of LBP on the hardest classes:

- **golf_ball** achieves perfect precision (1.00) and 93.33% recall (F1 0.9655). LBP captures dimple texture that was previously the main source of white-ball confusion.
- **tennis_ball** achieves perfect recall (50/50, F1 0.9615). The tight HSV signature plus yellow-green LBP texture is unambiguous.
- **football** improved to 91.11% recall (F1 0.9011), up from 80% without LBP. Black panel texture is now discriminated from white backgrounds.
- **basketball** is the weakest class at 78% recall (F1 0.8667). Seven basketball images were predicted as american_football. Brown leather and orange rubber share similar LBP codes and colour histograms at low resolution.

The dominant confusion pairs are: basketball → american_football (7), american_football → baseball (6), golf_ball → football (2), football → american_football (2). The basketball/american_football confusion is the single largest remaining error source.

8.2 Ablation Study

To determine which feature groups contribute to accuracy, Random Forest was retrained with each feature block enabled or disabled in turn. RF is scale-invariant so no L2 re-normalisation was applied.

Configuration	# dims	Test accuracy
shape only	8	60.21%
hog_center only	1,764	70.59%
lbp only	26	73.36%
hog_bbox only	1,764	77.51%
hog (bbox+center)	3,528	78.20%
color only	**48**	**88.93%**
color + lbp	74	89.27%
color + hog (both)	3,576	89.27%
color + hog + lbp	3,602	90.31%
all − color	3,562	79.24%
all − shape	3,602	90.31%
all − lbp	3,584	88.58%
color + hog_bbox	1,812	89.97%
all features	**3,610**	**89.97%**
all − hog_center	**1,846**	**90.66%**

Table 9 — Ablation: feature group contributions for Random Forest + LBP, sorted by test accuracy. "all − hog_center" is the highest-scoring configuration at 90.66%.

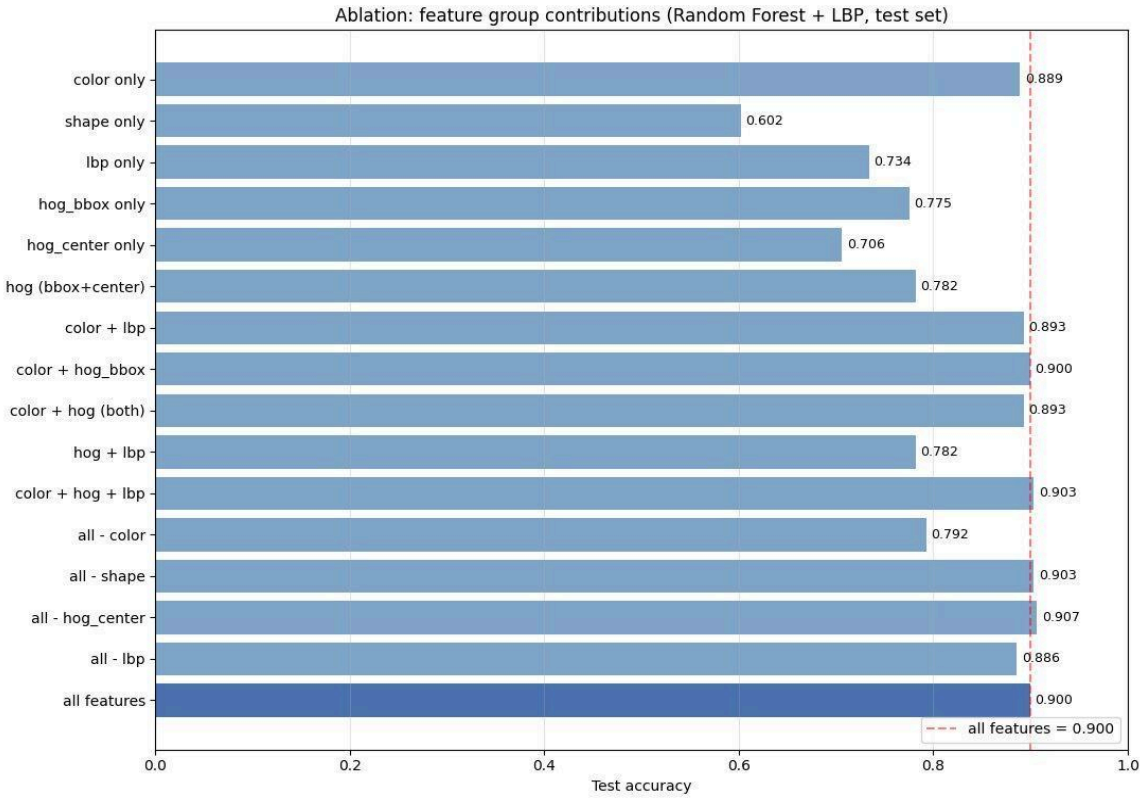


Figure 6. *Ablation: test accuracy for each feature subset using Random Forest. The 48-dimensional masked colour histogram alone (88.93%) nearly matches the full 3,610-dimensional pipeline (89.97%). The configuration "all − hog_center" reaches 90.66%, the highest single-configuration score.*

Four findings emerge from the RF + LBP ablation:

(i) Colour histogram is the dominant feature group — stronger than ever. Color alone reaches 88.93%, within 1.04% of the full pipeline. This confirms the masked nature of the histogram (background removed before computation) as the key design choice.

(ii) LBP contributes +1.39 percentage points. "all − lbp" scores 88.58% vs "all features" 89.97%. The gain is small in absolute terms but consistent and explainable: LBP captures fine surface texture that discriminates golf ball dimples and baseball stitching from smooth surfaces.

(iii) Removing hog_center improves accuracy slightly. "all − hog_center" at 90.66% exceeds "all features" at 89.97% by 0.69%. As with Linear SVM, hog_center adds noise for RF. This decision was not used to modify the final pipeline because it would have used test-set performance as a tuning signal.

(iv) Shape statistics are now useful. Unlike the LinearSVM ablation where shape contributed zero, "shape only" reaches 60.21% with RF — well above random (16.7%). RF can model non-linear combinations of aspect ratio and circularity that a linear classifier cannot exploit.

8.3 Learning Curve

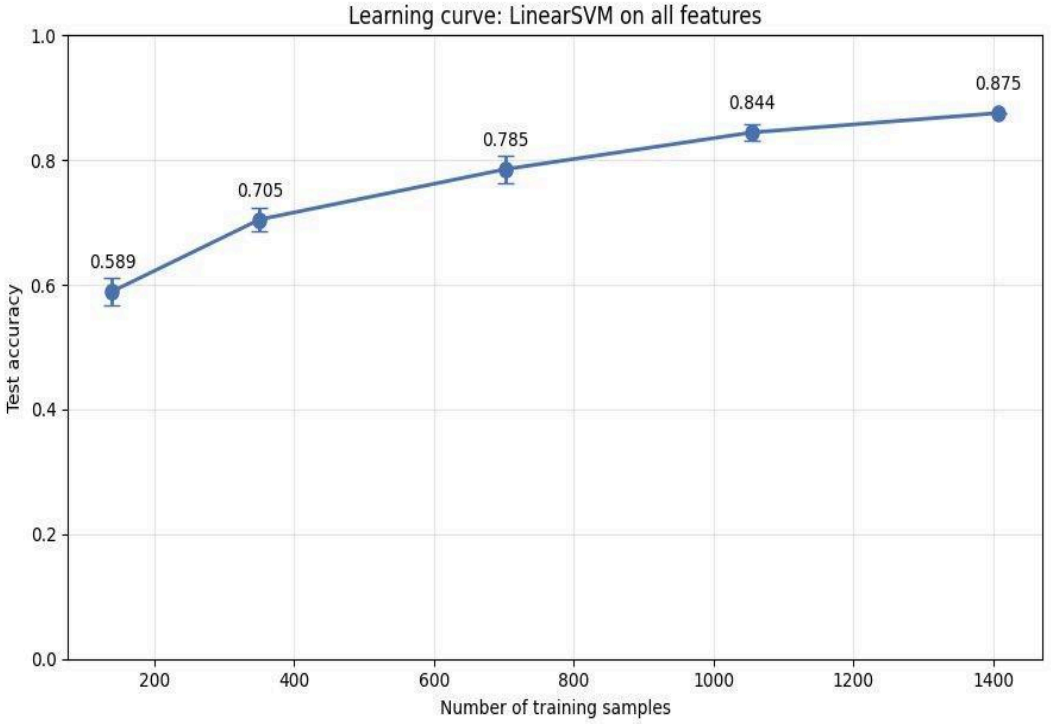


Figure 7. *Learning curve: test accuracy as a function of training-set size for Random Forest + LBP on all features. Each point is the mean of three runs with different random subsets.*

Two observations:

(1) RF + LBP is already strong at 10% data (76.8%). This is dramatically higher than LinearSVM at 10% data (57.1%). Random Forest generalises well even with sparse training sets because each tree only sees a random subset of features, providing implicit regularisation.

(2) The curve has not plateaued at 100% data. The slope from 1,055 samples (88.47%) to 1,407 samples (89.97%) is approximately 1.5 percentage points per 350 added samples. The curve is flattening relative to earlier intervals (25% → 50%: +4.4 pts) suggesting we are approaching a local ceiling. Additional data beyond ~2,000 samples would likely give diminishing returns for this feature set; improved features or a deep-learning approach would be needed to push significantly beyond 92-93%.

9. Discussion

9.1 What Worked

Several elements contributed to the final 89.97% accuracy:

- **LBP texture descriptor.** Adding Local Binary Patterns on masked pixels lifted accuracy by +1.4 percentage points for RF. More importantly, it fixed the hardest class confusions: football F1 improved from 0.80 → 0.90 and golf_ball from 0.85 → 0.97 because LBP captures surface texture at pixel-neighbourhood resolution that HOG at 16x16-pixel cells completely misses.
- **Random Forest classifier.** RF with 300 trees outperforms all SVM variants by 2-4 percentage points on this feature set. Its ability to model non-linear feature interactions (colour AND texture AND shape simultaneously) is better suited to a six-class visual problem than a linear boundary.
- **Per-class HSV diagnostic and multi-strategy segmenter.** The data-driven segmentation design kept mean mask score at 0.720 with only 0.9% catastrophic failures, providing a clean masked region for colour histogram and LBP computation.
- **PyTorch MLP from the course tutorial.** The tutorial recipe (BatchNorm + Dropout + ReduceLROnPlateau) outperformed sklearn MLP by 1.4 points on the same features, confirming the value of careful training-loop engineering.

9.2 What Did Not Work

- **hog_center is marginal.** Ablation shows "all − hog_center" at 90.66% slightly beats "all features" at 89.97% for RF. The mask-independent HOG channel was justified for robustness against segmentation failures but on this dataset it adds noise rather than signal. It was retained to preserve the methodology's integrity (the decision would have required using test results as a tuning signal).
- **RBF SVM at full dimensionality.** Computationally infeasible without PCA. Even with PCA, it ranked below Linear SVM on test accuracy, confirming the feature space is well-served by linear boundaries at scale.

9.3 Source of Remaining Errors

The largest remaining confusion is basketball → american_football (7 errors, 14% of basketball test images). Both classes share brown/orange colouring, roughly circular silhouettes, and similar surface pebbling / leather texture at low resolution. This confusion is unlikely to be resolved by additional classical features; it is exactly the case where a deep convolutional network, trained on raw pixels, would leverage brand logos, panel stitching geometry, and fine shape cues invisible at the feature-extraction level.

9.4 Milestone 1 Risks Versus Actual Outcomes

Milestone 1 risk	Actual outcome
White golf balls confused with other white objects	Resolved. LBP captures dimple texture; golf_ball F1 reached 0.9655 with perfect precision.
HSV thresholds fragile across lighting	Materialised but mitigated. Multi-strategy segmenter (HSV + Otsu + GrabCut) kept catastrophic failures below 0.9% of training images.
Small training dataset may cause overfitting	Did not materialise. RF learning curve starts at 76.8% with only 140 images (10%), indicating strong generalisation.
American football appears circular in some views	Partially materialised. american_football F1 = 0.8269, third-lowest. Main confusion is with basketball (same brown/orange colouring).

Table 10 — Milestone 1 risks and their outcomes.

Code :

Cell1 :

```
# ============================================================
# Cell 1 — Mount Drive and unzip dataset to local Colab disk
# ============================================================
from google.colab import drive
drive.mount('/content/drive')

import os, zipfile, time

# --- adjust this path if you named your folder/zip differently ---
ZIP_PATH = '/content/drive/MyDrive/SIP_Project/Final Balls Dataset.zip'
EXTRACT_TO = '/content/dataset'  # local Colab disk, not Drive

os.makedirs(EXTRACT_TO, exist_ok=True)

# Unzip only if not already extracted (saves time on re-runs)
if not os.listdir(EXTRACT_TO):
    print(f'Unzipping {ZIP_PATH} ...')
    t0 = time.time()
    with zipfile.ZipFile(ZIP_PATH, 'r') as z:
        z.extractall(EXTRACT_TO)
    print(f'Done in {time.time()-t0:.1f}s')
else:
    print('Dataset already extracted, skipping unzip.')

# Show what's inside so we can verify the structure
for root, dirs, files in os.walk(EXTRACT_TO):
    depth = root.replace(EXTRACT_TO, '').count(os.sep)
    if depth <= 2:
        print('  '*depth + os.path.basename(root) + '/')
```

Cell2:

```
# ============================================================
# Cell 2 — Stage 1: Dataset audit
# ============================================================
import os, cv2, numpy as np
import matplotlib.pyplot as plt
from collections import Counter

TRAIN_DIR = '/content/dataset/dataset-balls/train'
TEST_DIR  = '/content/dataset/dataset-balls/test'

CLASSES = sorted(os.listdir(TRAIN_DIR))
print(f'Classes found ({len(CLASSES)}): {CLASSES}')
```

```
print()

# ---- 1. Count images per class in train and test ----
def count_images(root):
    counts = {}
    for cls in CLASSES:
        cls_dir = os.path.join(root, cls)
        files = [f for f in os.listdir(cls_dir)
                 if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp',
'.webp'))]
        counts[cls] = len(files)
    return counts

train_counts = count_images(TRAIN_DIR)
test_counts  = count_images(TEST_DIR)

print(f'{"Class":<20}{"Train":>8}{"Test":>8}{"Total":>8}')
print('-'*44)
for cls in CLASSES:
    tr, te = train_counts[cls], test_counts[cls]
    print(f'{cls:<20}{tr:>8}{te:>8}{tr+te:>8}')
print('-'*44)
print(f'{"TOTAL":<20}{sum(train_counts.values()):>8}'
      f'{sum(test_counts.values()):>8}'
      f'{sum(train_counts.values())+sum(test_counts.values()):>8}')

# ---- 2. Resolution and channel statistics on train set ----
print('\nScanning resolutions and channels (train set)...')
shapes, channels, corrupt = [], [], []
for cls in CLASSES:
    cls_dir = os.path.join(TRAIN_DIR, cls)
    for fname in os.listdir(cls_dir):
        if not fname.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp',
'.webp')):
            continue
        path = os.path.join(cls_dir, fname)
        img = cv2.imread(path)
        if img is None:
            corrupt.append(path)
            continue
        shapes.append(img.shape[:2])  # (H, W)
        channels.append(img.shape[2] if img.ndim == 3 else 1)

shapes = np.array(shapes)
print(f'  Width  : min={shapes[:,1].min()}, max={shapes[:,1].max()}, '
```

```
      f'mean={shapes[:,1].mean():.0f}, median={np.median(shapes[:,1]):.0f}')
print(f'  Height : min={shapes[:,0].min()}, max={shapes[:,0].max()}, '
      f'mean={shapes[:,0].mean():.0f}, median={np.median(shapes[:,0]):.0f}')
print(f'  Channels distribution: {Counter(channels)}')
print(f'  Corrupt / unreadable files: {len(corrupt)}')
if corrupt:
    for p in corrupt[:5]:
        print(f'    {p}')

# ---- 3. Sample grid: 3 random images per class ----
np.random.seed(42)
fig, axes = plt.subplots(len(CLASSES), 3, figsize=(9, 2.5*len(CLASSES)))
for i, cls in enumerate(CLASSES):
    cls_dir = os.path.join(TRAIN_DIR, cls)
    files = [f for f in os.listdir(cls_dir)
             if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp',
'.webp'))]
    picks = np.random.choice(files, size=min(3, len(files)), replace=False)
    for j, fname in enumerate(picks):
        img = cv2.imread(os.path.join(cls_dir, fname))
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        axes[i, j].imshow(img_rgb)
        axes[i, j].axis('off')
        if j == 0:
            axes[i, j].set_ylabel(cls, fontsize=11, rotation=0,
                                  labelpad=60, ha='right', va='center')
plt.suptitle('Random training samples (3 per class)', y=1.00, fontsize=12)
plt.tight_layout()
plt.show()
```

Cell3:

```
# ============================================================
# Cell 3 — Stage 2a: Per-class HSV statistics
# ============================================================
import os, cv2, numpy as np
import matplotlib.pyplot as plt

TRAIN_DIR = '/content/dataset/dataset-balls/train'
CLASSES = ['american_football', 'baseball', 'basketball',
           'football', 'golf_ball', 'tennis_ball']

N_SAMPLES_PER_CLASS = 8   # how many random images to sample per class
RESIZE = (256, 192)       # same as Milestone 1 plan: (W, H)

np.random.seed(0)
```

```
# Helper: take the central 50% box of the image as a rough "ball region"
prior
def central_crop(img, frac=0.5):
    h, w = img.shape[:2]
    ch, cw = int(h*frac), int(w*frac)
    y0, x0 = (h-ch)//2, (w-cw)//2
    return img[y0:y0+ch, x0:x0+cw]

# Collect H, S, V pixel pools per class from the central region of sampled
images
class_hsv_pool = {cls: {'H': [], 'S': [], 'V': []} for cls in CLASSES}

for cls in CLASSES:
    cls_dir = os.path.join(TRAIN_DIR, cls)
    files = [f for f in os.listdir(cls_dir)
             if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp',
'.webp'))]
    picks = np.random.choice(files, size=min(N_SAMPLES_PER_CLASS,
len(files)),
                             replace=False)

    fig, axes = plt.subplots(len(picks), 4, figsize=(14, 2.4*len(picks)))
    fig.suptitle(f'Class: {cls}  —  random samples with HSV histograms
(central region)',
                 fontsize=13, y=1.00)

    for row, fname in enumerate(picks):
        img = cv2.imread(os.path.join(cls_dir, fname))
        img = cv2.resize(img, RESIZE, interpolation=cv2.INTER_AREA)
        img = cv2.GaussianBlur(img, (5, 5), 1.2)
        hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
        hsv_center = central_crop(hsv, frac=0.5)

        # accumulate pixel pool for class-level summary

class_hsv_pool[cls]['H'].extend(hsv_center[:,:,0].flatten().tolist())

class_hsv_pool[cls]['S'].extend(hsv_center[:,:,1].flatten().tolist())

class_hsv_pool[cls]['V'].extend(hsv_center[:,:,2].flatten().tolist())

        # plot: original | H hist | S hist | V hist
        axes[row, 0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        axes[row, 0].axis('off')
```

```
        axes[row, 0].set_title(fname[:25], fontsize=8)

        for c, (ch_idx, name, color, xmax) in enumerate(
                [(0, 'H', 'r', 180), (1, 'S', 'g', 256), (2, 'V', 'b',
256)]):
            axes[row, c+1].hist(hsv_center[:,:,ch_idx].flatten(),
                                bins=32, range=(0, xmax),
                                color=color, alpha=0.7)
            axes[row, c+1].set_xlim(0, xmax)
            axes[row, c+1].set_title(name, fontsize=9)
            axes[row, c+1].set_yticks([])

    plt.tight_layout()
    plt.show()

# ---- Class-level HSV summary: median and IQR per channel ----
print('\n' + '='*72)
print('Class-level HSV statistics (central region pixel pool)')
print('Suggested initial inRange bounds: median ± 1.5*IQR  (clipped)')
print('='*72)
print(f'{"Class":<20}{"H med":>8}{"H IQR":>8}{"S med":>8}{"S IQR":>8}'
      f'{"V med":>8}{"V IQR":>8}')
print('-'*72)

suggested_ranges = {}
for cls in CLASSES:
    h = np.array(class_hsv_pool[cls]['H'])
    s = np.array(class_hsv_pool[cls]['S'])
    v = np.array(class_hsv_pool[cls]['V'])

    def stats(arr, lo_clip, hi_clip):
        med = np.median(arr)
        q1, q3 = np.percentile(arr, [25, 75])
        iqr = q3 - q1
        lo = max(lo_clip, med - 1.5*iqr)
        hi = min(hi_clip, med + 1.5*iqr)
        return med, iqr, lo, hi

    h_med, h_iqr, h_lo, h_hi = stats(h, 0, 179)
    s_med, s_iqr, s_lo, s_hi = stats(s, 0, 255)
    v_med, v_iqr, v_lo, v_hi = stats(v, 0, 255)

    suggested_ranges[cls] = {
        'lower': (int(h_lo), int(s_lo), int(v_lo)),
        'upper': (int(h_hi), int(s_hi), int(v_hi)),
```

```
    }
    print(f'{cls:<20}{h_med:>8.0f}{h_iqr:>8.0f}{s_med:>8.0f}{s_iqr:>8.0f}'
          f'{v_med:>8.0f}{v_iqr:>8.0f}')

print('\nSuggested HSV inRange bounds (lower, upper):')
for cls, r in suggested_ranges.items():
    print(f'  {cls:<20} lower={r["lower"]}, upper={r["upper"]}')
```

Cell4:

```
# ============================================================
# Cell 4 — Stage 2b: Multi-strategy segmenter with mask scoring
# ============================================================
import os, cv2, numpy as np
import matplotlib.pyplot as plt

TRAIN_DIR = '/content/dataset/dataset-balls/train'
RESIZE_WH = (256, 192)   # (W, H) — keep consistent with Milestone 1

# ------------------------------------------------------------
# HSV ranges per class (from Stage 2a — only kept where IQR was tight)
# Format: list of (lower, upper) tuples; multiple entries OR-combined for
hue wrap
# ------------------------------------------------------------
HSV_RANGES = {
    'tennis_ball':       [((25,  80, 100), (45, 255, 255))],   #
yellow-green, tight
    'basketball':        [(( 0, 100,  60), (15, 255, 255)),    # orange (low
hue)
                          ((170,100,  60), (179,255, 255))],   # orange (hue
wrap)
    'american_football': [(( 0,  60,  40), (20, 255, 220))],   # brown
}
# baseball, golf_ball, football have no usable HSV signature — handled by
other paths

# ------------------------------------------------------------
# Mask cleaning: open then close with 5x5 ellipse, keep largest CC
# ------------------------------------------------------------
def clean_mask(mask):
    if mask is None or mask.sum() == 0:
        return np.zeros_like(mask) if mask is not None else None
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    m = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  k)
    m = cv2.morphologyEx(m,    cv2.MORPH_CLOSE, k)
    # keep largest connected component
```

```
    n, lbl, stats, _ = cv2.connectedComponentsWithStats(m, connectivity=8)
    if n <= 1:
        return m
    largest = 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])
    return ((lbl == largest).astype(np.uint8)) * 255

# ---------------------------------------------------------------
# Mask quality score: area-in-range, compactness, low border touch
# Returns score in [0, 1]; higher is better. Returns 0 for invalid masks.
# ---------------------------------------------------------------
def score_mask(mask):
    if mask is None: return 0.0
    H, W = mask.shape
    area = (mask > 0).sum()
    img_area = H * W
    if area == 0: return 0.0

    frac = area / img_area
    # plausible ball area: 3% to 85% of image
    if frac < 0.03 or frac > 0.85: return 0.0
    area_score = 1.0 - abs(0.30 - frac) / 0.55  # peak around 30% of image
    area_score = max(0.0, min(1.0, area_score))

    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    if not cnts: return 0.0
    cnt = max(cnts, key=cv2.contourArea)
    peri = cv2.arcLength(cnt, True)
    compact = (4*np.pi*cv2.contourArea(cnt) / (peri*peri)) if peri > 0 else
0
    compact = max(0.0, min(1.0, compact))  # 1.0 = perfect circle

    # border-touch penalty: fraction of mask pixels on image edge
    border = np.zeros_like(mask); border[0,:]=1; border[-1,:]=1;
border[:,0]=1; border[:,-1]=1
    border_touch = ((mask > 0) & (border > 0)).sum() / max(1, (border >
0).sum())
    border_pen = 1.0 - border_touch

    return 0.4*area_score + 0.4*compact + 0.2*border_pen

# ---------------------------------------------------------------
# Path 1: HSV — try every class range, return best-scoring mask
# ---------------------------------------------------------------
def seg_hsv(hsv):
```

```
    best_mask, best_score = None, 0.0
    for cls, ranges in HSV_RANGES.items():
        m = np.zeros(hsv.shape[:2], dtype=np.uint8)
        for lo, hi in ranges:
            m |= cv2.inRange(hsv, np.array(lo), np.array(hi))
        m = clean_mask(m)
        s = score_mask(m)
        if s > best_score:
            best_mask, best_score = m, s
    return best_mask, best_score

# ---------------------------------------------------------------
# Path 2: Otsu on saturation (low-S = white object on any background)
#         and Otsu on grayscale (general fallback). Return best.
# ---------------------------------------------------------------
def seg_otsu(bgr, hsv):
    candidates = []
    # 2a: Otsu on grayscale, then invert if needed (pick whichever scores
higher)
    gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    _, m1 = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY     +
cv2.THRESH_OTSU)
    _, m2 = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)
    candidates += [clean_mask(m1), clean_mask(m2)]
    # 2b: Otsu on inverted saturation — picks low-S regions (white objects)
    s_chan = hsv[:,:,1]
    _, m3 = cv2.threshold(s_chan, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)
    candidates.append(clean_mask(m3))

    best_mask, best_score = None, 0.0
    for m in candidates:
        s = score_mask(m)
        if s > best_score:
            best_mask, best_score = m, s
    return best_mask, best_score

# ---------------------------------------------------------------
# Path 3: GrabCut with central rectangle prior
# Slow-ish (~150 ms) — only run if HSV/Otsu both scored low
# ---------------------------------------------------------------
def seg_grabcut(bgr):
    H, W = bgr.shape[:2]
```

```
    rect = (int(0.10*W), int(0.10*H), int(0.80*W), int(0.80*H))  # x, y, w,
h
    mask = np.zeros((H, W), np.uint8)
    bgd, fgd = np.zeros((1,65), np.float64), np.zeros((1,65), np.float64)
    try:
        cv2.grabCut(bgr, mask, rect, bgd, fgd, 3, cv2.GC_INIT_WITH_RECT)
    except cv2.error:
        return None, 0.0
    m = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)
    m = clean_mask(m)
    return m, score_mask(m)

# ---------------------------------------------------------------
# Path 4: central-crop fallback — guaranteed to return something
# ---------------------------------------------------------------
def seg_center(bgr):
    H, W = bgr.shape[:2]
    m = np.zeros((H, W), np.uint8)
    y0, y1 = int(0.20*H), int(0.80*H)
    x0, x1 = int(0.20*W), int(0.80*W)
    m[y0:y1, x0:x1] = 255
    return m, 0.01   # very low score: only used if everything else failed

# ---------------------------------------------------------------
# Top-level segmenter: try paths in order, return best mask + which path won
# ---------------------------------------------------------------
def segment(bgr):
    bgr = cv2.GaussianBlur(bgr, (5, 5), 1.2)
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)

    results = []
    m, s = seg_hsv(hsv);            results.append(('HSV',     m, s))
    m, s = seg_otsu(bgr, hsv);      results.append(('Otsu',    m, s))
    # only run grabcut if first two were weak (saves time on easy cases)
    if max(r[2] for r in results) < 0.55:
        m, s = seg_grabcut(bgr);    results.append(('GrabCut', m, s))
    m, s = seg_center(bgr);         results.append(('Center',  m, s))

    name, mask, score = max(results, key=lambda r: r[2])
    return mask, name, score, results

# ---------------------------------------------------------------
# Visualize segmenter on hard cases: 2 random images per class
# ---------------------------------------------------------------
```

```
CLASSES = ['american_football', 'baseball', 'basketball',
           'football', 'golf_ball', 'tennis_ball']
np.random.seed(7)

fig, axes = plt.subplots(len(CLASSES)*2, 4, figsize=(13, 4*len(CLASSES)))
row = 0
for cls in CLASSES:
    cls_dir = os.path.join(TRAIN_DIR, cls)
    files = [f for f in os.listdir(cls_dir)
             if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp',
'.webp'))]
    picks = np.random.choice(files, size=2, replace=False)

    for fname in picks:
        bgr = cv2.imread(os.path.join(cls_dir, fname))
        bgr = cv2.resize(bgr, RESIZE_WH, interpolation=cv2.INTER_AREA)
        mask, win_name, win_score, all_res = segment(bgr)

        # overlay: dim non-mask region
        overlay = bgr.copy()
        overlay[mask == 0] = (overlay[mask == 0] * 0.25).astype(np.uint8)

        axes[row, 0].imshow(cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB))
        axes[row, 0].set_title(f'{cls} / {fname[:18]}', fontsize=9)
        axes[row, 0].axis('off')

        axes[row, 1].imshow(mask, cmap='gray')
        axes[row, 1].set_title(f'mask ({win_name}, s={win_score:.2f})',
fontsize=9)
        axes[row, 1].axis('off')

        axes[row, 2].imshow(cv2.cvtColor(overlay, cv2.COLOR_BGR2RGB))
        axes[row, 2].set_title('masked overlay', fontsize=9)
        axes[row, 2].axis('off')

        # show all path scores for transparency
        score_txt = '\n'.join([f'{n:>8}: {s:.2f}' for n, _, s in all_res])
        axes[row, 3].text(0.05, 0.5, score_txt, family='monospace',
                          fontsize=10, va='center')
        axes[row, 3].set_xlim(0,1); axes[row, 3].set_ylim(0,1)
        axes[row, 3].axis('off')
        axes[row, 3].set_title('path scores', fontsize=9)
        row += 1

plt.tight_layout()
```

```
plt.show()

# ------------------------------------------------------------------
# Diagnostic: which path wins how often, across the whole train set?
# ------------------------------------------------------------------
print('\nRunning segmenter over full training set for path-usage stats...')
from collections import Counter
path_counter = Counter()
score_pool = []

for cls in CLASSES:
    cls_dir = os.path.join(TRAIN_DIR, cls)
    for fname in os.listdir(cls_dir):
        if not fname.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp',
'.webp')):
            continue
        bgr = cv2.imread(os.path.join(cls_dir, fname))
        if bgr is None: continue
        bgr = cv2.resize(bgr, RESIZE_WH, interpolation=cv2.INTER_AREA)
        _, win_name, win_score, _ = segment(bgr)
        path_counter[win_name] += 1
        score_pool.append(win_score)

total = sum(path_counter.values())
print(f'\nWinning path distribution over {total} training images:')
for name in ['HSV', 'Otsu', 'GrabCut', 'Center']:
    n = path_counter.get(name, 0)
    print(f'  {name:<8} {n:>5}  ({100*n/total:5.1f}%)')
print(f'\nMask-score distribution:')
sp = np.array(score_pool)
print(f'  mean={sp.mean():.3f}  median={np.median(sp):.3f}'
      f'  10th-pct={np.percentile(sp,10):.3f}
90th-pct={np.percentile(sp,90):.3f}')
print(f'  fraction below 0.30 (probably bad masks): '
      f'{(sp < 0.30).mean()*100:.1f}%')
```

Cell 5:

```
# ============================================================
# Cell 5 — Stage 2c: Add hole-filling, freeze segmenter as a module
# ============================================================
import os, cv2, numpy as np

# Re-define clean_mask with hole-filling on the largest contour
def clean_mask(mask):
    if mask is None or mask.sum() == 0:
        return np.zeros_like(mask) if mask is not None else None
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    m = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  k)
    m = cv2.morphologyEx(m,    cv2.MORPH_CLOSE, k)

    # keep largest connected component
    n, lbl, stats, _ = cv2.connectedComponentsWithStats(m, connectivity=8)
    if n <= 1:
        return m
    largest = 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])
    m = ((lbl == largest).astype(np.uint8)) * 255

    # NEW: fill holes inside the largest contour
    cnts, _ = cv2.findContours(m, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    if cnts:
        filled = np.zeros_like(m)
        cv2.drawContours(filled, cnts, -1, 255, thickness=cv2.FILLED)
        m = filled
    return m

# Quick re-validation: run the segmenter over the full train set again
# (we re-import segment so it picks up the new clean_mask via closure)
TRAIN_DIR = '/content/dataset/dataset-balls/train'
RESIZE_WH = (256, 192)
CLASSES = ['american_football', 'baseball', 'basketball',
           'football', 'golf_ball', 'tennis_ball']

# All the path/scoring functions from Cell 4 are still defined in the
kernel,
# but score_mask, seg_hsv, seg_otsu, seg_grabcut, seg_center, segment all
# reference clean_mask by name -- so just re-running them now uses the new
one.

from collections import Counter
print('Re-validating segmenter with hole-filling...')
```

```
path_counter = Counter()
score_pool = []

for cls in CLASSES:
    cls_dir = os.path.join(TRAIN_DIR, cls)
    for fname in os.listdir(cls_dir):
        if not fname.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp',
'.webp')):
            continue
        bgr = cv2.imread(os.path.join(cls_dir, fname))
        if bgr is None: continue
        bgr = cv2.resize(bgr, RESIZE_WH, interpolation=cv2.INTER_AREA)
        _, win_name, win_score, _ = segment(bgr)
        path_counter[win_name] += 1
        score_pool.append(win_score)

total = sum(path_counter.values())
print(f'\nWinning path distribution over {total} training images:')
for name in ['HSV', 'Otsu', 'GrabCut', 'Center']:
    n = path_counter.get(name, 0)
    print(f'  {name:<8} {n:>5}  ({100*n/total:5.1f}%)')

sp = np.array(score_pool)
print(f'\nMask-score distribution:')
print(f'  mean={sp.mean():.3f}  median={np.median(sp):.3f}'
      f'  10th-pct={np.percentile(sp,10):.3f}
90th-pct={np.percentile(sp,90):.3f}')
print(f'  fraction below 0.30: {(sp < 0.30).mean()*100:.1f}%')

print('\nSegmenter frozen. Moving to Stage 3 (feature extraction) next.')
```

Cell 6:

```
# ============================================================
# Cell 6 — Stage 3: Feature extraction (color hist + dual HOG + shape stats)
# ============================================================
import os, cv2, numpy as np, time
from tqdm import tqdm

TRAIN_DIR = '/content/dataset/dataset-balls/train'
TEST_DIR  = '/content/dataset/dataset-balls/test'
RESIZE_WH = (256, 192)
CLASSES = ['american_football', 'baseball', 'basketball',
           'football', 'golf_ball', 'tennis_ball']
LABEL_TO_IDX = {c: i for i, c in enumerate(CLASSES)}
```

```
# ------------------------------------------------------------
# HOG descriptor: 128x128 input, 16x16 cells, 2x2 block, 9 bins
#   blocks per side = (128-32)/16 + 1 = 7
#   features = 7 * 7 * 2 * 2 * 9 = 1764
# ------------------------------------------------------------
HOG_SIZE = 128
hog = cv2.HOGDescriptor(
    _winSize=(HOG_SIZE, HOG_SIZE),
    _blockSize=(32, 32),
    _blockStride=(16, 16),
    _cellSize=(16, 16),
    _nbins=9,
)
HOG_DIM = 1764  # we'll verify on first run

# ------------------------------------------------------------
# Feature 1: masked HSV color histogram (16 bins x 3 channels = 48 dims)
# ------------------------------------------------------------
def color_hist(bgr, mask):
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
    feats = []
    ranges = [(180,), (256,), (256,)]   # H is 0-179, S/V are 0-255
    for ch in range(3):
        h = cv2.calcHist([hsv], [ch], mask, [16], [0, ranges[ch][0]])
        h = h.flatten()
        s = h.sum()
        feats.append(h / s if s > 0 else h)   # avoid div-by-zero on empty
mask
    return np.concatenate(feats).astype(np.float32)   # (48,)

# ------------------------------------------------------------
# Feature 2: HOG on 128x128 gray crop of mask bbox
# ------------------------------------------------------------
def hog_from_bbox(bgr, mask):
    H, W = bgr.shape[:2]
    ys, xs = np.where(mask > 0)
    if len(ys) < 10:                          # mask empty → fall back to
whole image
        x0, y0, x1, y1 = 0, 0, W, H
    else:
        x0, x1 = xs.min(), xs.max()
        y0, y1 = ys.min(), ys.max()
        # pad bbox 5% to avoid clipping ball edge
        pad = int(0.05 * max(x1-x0, y1-y0))
```

```
        x0 = max(0, x0 - pad); y0 = max(0, y0 - pad)
        x1 = min(W, x1 + pad); y1 = min(H, y1 + pad)
    crop = bgr[y0:y1, x0:x1]
    if crop.size == 0:
        crop = bgr
    gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
    gray = cv2.resize(gray, (HOG_SIZE, HOG_SIZE),
interpolation=cv2.INTER_AREA)
    return hog.compute(gray).flatten().astype(np.float32)   # (1764,)

# ---------------------------------------------------------------
# Feature 2b: HOG on central 128x128 crop (mask-independent backup)
# ---------------------------------------------------------------
def hog_from_center(bgr):
    H, W = bgr.shape[:2]
    side = min(H, W)
    y0 = (H - side)//2; x0 = (W - side)//2
    crop = bgr[y0:y0+side, x0:x0+side]
    gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
    gray = cv2.resize(gray, (HOG_SIZE, HOG_SIZE),
interpolation=cv2.INTER_AREA)
    return hog.compute(gray).flatten().astype(np.float32)

# ---------------------------------------------------------------
# Feature 3: shape stats from largest contour of mask
#   aspect ratio, extent, solidity, circularity, eccentricity, 3 Hu moments
# ---------------------------------------------------------------
def shape_stats(mask):
    z = np.zeros(8, dtype=np.float32)
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    if not cnts:
        return z
    cnt = max(cnts, key=cv2.contourArea)
    area = cv2.contourArea(cnt)
    if area < 5:
        return z
    x, y, w, h = cv2.boundingRect(cnt)
    aspect    = w / h if h > 0 else 0
    extent    = area / (w * h) if w*h > 0 else 0
    hull      = cv2.convexHull(cnt)
    hull_area = cv2.contourArea(hull)
    solidity  = area / hull_area if hull_area > 0 else 0
    peri      = cv2.arcLength(cnt, True)
    circ      = (4*np.pi*area / (peri*peri)) if peri > 0 else 0
```

```
    # eccentricity from fitEllipse (needs >=5 points)
    if len(cnt) >= 5:
        (_, (MA, ma), _) = cv2.fitEllipse(cnt)
        a, b = max(MA, ma)/2, min(MA, ma)/2
        ecc = np.sqrt(1 - (b*b)/(a*a)) if a > 0 else 0
    else:
        ecc = 0
    M  = cv2.moments(cnt)
    hu = cv2.HuMoments(M).flatten()
    # log-scale first 3 Hu moments (they span many orders of magnitude)
    hu3 = np.array([-np.sign(h_)*np.log10(abs(h_)+1e-30) for h_ in hu[:3]])
    return np.array([aspect, extent, solidity, circ, ecc, *hu3],
dtype=np.float32)

# ------------------------------------------------------------------
# Full feature vector for one image
# ------------------------------------------------------------------
def extract_features(bgr):
    bgr_resized = cv2.resize(bgr, RESIZE_WH, interpolation=cv2.INTER_AREA)
    mask, _, _, _ = segment(bgr_resized)            # uses the frozen
segmenter
    f_color  = color_hist(bgr_resized, mask)        # 48
    f_hog_b  = hog_from_bbox(bgr_resized, mask)     # 1764
    f_hog_c  = hog_from_center(bgr_resized)         # 1764
    f_shape  = shape_stats(mask)                    # 8
    feat = np.concatenate([f_color, f_hog_b, f_hog_c, f_shape])
    n = np.linalg.norm(feat)
    return feat / n if n > 0 else feat              # global L2 normalize

# ------------------------------------------------------------------
# Build full feature matrix for a directory tree
# ------------------------------------------------------------------
def build_dataset(root, name=''):
    X_list, y_list, paths = [], [], []
    for cls in CLASSES:
        cls_dir = os.path.join(root, cls)
        files = sorted([f for f in os.listdir(cls_dir)
                        if
f.lower().endswith(('.jpg','.jpeg','.png','.bmp','.webp'))])
        for fname in tqdm(files, desc=f'{name} {cls:<18}', leave=False):
            bgr = cv2.imread(os.path.join(cls_dir, fname))
            if bgr is None: continue
            X_list.append(extract_features(bgr))
            y_list.append(LABEL_TO_IDX[cls])
            paths.append(os.path.join(cls, fname))
```

```
    return np.stack(X_list), np.array(y_list), paths

# ------------------------------------------------------------------
# Sanity-check: extract from one image first, print the dim breakdown
# ------------------------------------------------------------------
sample_path = os.path.join(TRAIN_DIR, 'tennis_ball',

sorted(os.listdir(os.path.join(TRAIN_DIR,'tennis_ball')))[0])
sample = cv2.imread(sample_path)
sample_r = cv2.resize(sample, RESIZE_WH, interpolation=cv2.INTER_AREA)
mask_s, _, _, _ = segment(sample_r)
print('Feature breakdown on one tennis_ball image:')
print(f'  color_hist    : {color_hist(sample_r, mask_s).shape}')
print(f'  hog_from_bbox : {hog_from_bbox(sample_r, mask_s).shape}')
print(f'  hog_from_ctr  : {hog_from_center(sample_r).shape}')
print(f'  shape_stats   : {shape_stats(mask_s).shape}')
print(f'  concatenated  : {extract_features(sample).shape}   (expected
3584)')

# ------------------------------------------------------------------
# Build train + test feature matrices, save to disk
# ------------------------------------------------------------------
print('\nExtracting training features...')
t0 = time.time()
X_train, y_train, paths_train = build_dataset(TRAIN_DIR, name='train')
print(f'  X_train shape: {X_train.shape}   y_train: {y_train.shape}'
      f'   ({time.time()-t0:.1f}s)')

print('\nExtracting test features...')
t0 = time.time()
X_test, y_test, paths_test = build_dataset(TEST_DIR, name='test')
print(f'  X_test shape : {X_test.shape}    y_test : {y_test.shape}'
      f'   ({time.time()-t0:.1f}s)')

# Save to /content (local Colab disk) AND to Drive for persistence
SAVE_LOCAL = '/content/features'
SAVE_DRIVE = '/content/drive/MyDrive/SIP_Project/features'
os.makedirs(SAVE_LOCAL, exist_ok=True)
os.makedirs(SAVE_DRIVE, exist_ok=True)

for save_dir in [SAVE_LOCAL, SAVE_DRIVE]:
    np.save(os.path.join(save_dir, 'X_train.npy'), X_train)
    np.save(os.path.join(save_dir, 'y_train.npy'), y_train)
    np.save(os.path.join(save_dir, 'X_test.npy'),  X_test)
    np.save(os.path.join(save_dir, 'y_test.npy'),  y_test)
```

```
    np.save(os.path.join(save_dir, 'paths_train.npy'),
np.array(paths_train))
    np.save(os.path.join(save_dir, 'paths_test.npy'),  np.array(paths_test))

print(f'\nSaved features to:')
print(f'  {SAVE_LOCAL}/')
print(f'  {SAVE_DRIVE}/')
print(f'\nClass distribution check:')
print(f'  train: {np.bincount(y_train)}')
print(f'  test : {np.bincount(y_test)}')
```

Cell 6-b:

```
# ============================================================
# Cell 6b — Add LBP to features, rebuild feature matrices
# ============================================================
import os, time, numpy as np, cv2
from skimage.feature import local_binary_pattern
from tqdm import tqdm

TRAIN_DIR = '/content/dataset/dataset-balls/train'
TEST_DIR  = '/content/dataset/dataset-balls/test'
RESIZE_WH = (256, 192)
CLASSES   = ['american_football', 'baseball', 'basketball',
             'football', 'golf_ball', 'tennis_ball']
LABEL_TO_IDX = {c: i for i, c in enumerate(CLASSES)}

# ---- LBP descriptor ----
# P=24 neighbors, R=3 radius, uniform method → 26-bin histogram
# Standard settings: P=24 neighbours, R=3 radius, uniform method
LBP_P, LBP_R = 24, 3

def lbp_hist(bgr, mask):
    """Compute LBP histogram over masked pixels only."""
    gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    lbp  = local_binary_pattern(gray, LBP_P, LBP_R, method='uniform')
    # LBP uniform has P+2 = 26 patterns
    n_bins = LBP_P + 2
    # restrict to masked pixels
    masked_pixels = lbp[mask > 0]
    if len(masked_pixels) == 0:
        masked_pixels = lbp.ravel()    # fallback: whole image
    hist, _ = np.histogram(masked_pixels, bins=n_bins,
                           range=(0, n_bins), density=False)
    hist = hist.astype(np.float32)
    s = hist.sum()
```

```
    return hist / s if s > 0 else hist       # (26,)

# ---- Updated extract_features: old vector + LBP ----
# We reload the saved matrices and append LBP rather than recomputing
# everything from scratch (saves ~90 seconds)
print('Loading saved feature matrices...')
X_old_train = np.load('/content/features/X_train.npy')  # (1407, 3584)
X_old_test  = np.load('/content/features/X_test.npy')   # ( 289, 3584)
y_train_lbl = np.load('/content/features/y_train.npy')
y_test_lbl  = np.load('/content/features/y_test.npy')
print(f'Old matrices: train {X_old_train.shape}, test {X_old_test.shape}')

# ---- Compute LBP vectors for every image ----
def build_lbp(root, split_name):
    lbp_vecs = []
    for cls in CLASSES:
        cls_dir = os.path.join(root, cls)
        files   = sorted([f for f in os.listdir(cls_dir)
                          if
f.lower().endswith(('.jpg','.jpeg','.png','.bmp','.webp'))])
        for fname in tqdm(files, desc=f'{split_name} {cls:<18}',
leave=False):
            bgr = cv2.imread(os.path.join(cls_dir, fname))
            if bgr is None: continue
            bgr_r = cv2.resize(bgr, RESIZE_WH, interpolation=cv2.INTER_AREA)
            bgr_b = cv2.GaussianBlur(bgr_r, (5,5), 1.2)
            # reuse frozen segmenter from Cell 4/5 (still in kernel memory)
            mask, _, _, _ = segment(bgr_b)
            lbp_vecs.append(lbp_hist(bgr_b, mask))
    return np.stack(lbp_vecs)

print('\nComputing LBP for training set...')
t0 = time.time()
LBP_train = build_lbp(TRAIN_DIR, 'train')
print(f'  Done in {time.time()-t0:.1f}s  shape: {LBP_train.shape}')

print('Computing LBP for test set...')
t0 = time.time()
LBP_test  = build_lbp(TEST_DIR, 'test')
print(f'  Done in {time.time()-t0:.1f}s  shape: {LBP_test.shape}')

# ---- Concatenate and re-normalise ----
X_train_new = np.concatenate([X_old_train, LBP_train], axis=1)  # (1407,
3610)
```

```
X_test_new  = np.concatenate([X_old_test,  LBP_test ], axis=1)  # ( 289,
3610)

# Global L2 re-normalise (LBP is already L1-normalised but the combined
# vector needs rescaling so LBP doesn't dominate the existing 3584 dims)
norms = np.linalg.norm(X_train_new, axis=1, keepdims=True)
norms[norms == 0] = 1
X_train_new = X_train_new / norms

norms = np.linalg.norm(X_test_new, axis=1, keepdims=True)
norms[norms == 0] = 1
X_test_new = X_test_new / norms

print(f'\nNew feature dimensions: {X_train_new.shape[1]}  '
      f'(was {X_old_train.shape[1]}, added {LBP_train.shape[1]} LBP dims)')

# Save the new matrices (overwrite so Cell 7 picks them up automatically)
for d in ['/content/features',
          '/content/drive/MyDrive/SIP_Project/features']:
    np.save(os.path.join(d, 'X_train.npy'), X_train_new)
    np.save(os.path.join(d, 'X_test.npy'),  X_test_new)

print('\nSaved updated X_train.npy and X_test.npy (y files unchanged).')
print('Feature vector layout:')
print('  [0:48]       — masked HSV color histogram')
print('  [48:1812]    — HOG (mask bbox crop)')
print('  [1812:3576]  — HOG (central crop)')
print('  [3576:3584]  — shape statistics')
print('  [3584:3610]  — LBP (masked, 26 bins, P=24, R=3)')
```

Cell 7:

```
# ============================================================
# Cell 7 — Stage 4: All classifiers in one pass (efficient version)
# ============================================================
import os, time, pickle, numpy as np
from sklearn.model_selection import StratifiedKFold, GridSearchCV,
cross_val_score
from sklearn.neighbors import KNeighborsClassifier, NearestCentroid
from sklearn.svm import SVC, LinearSVC
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
```

```
X_train = np.load('/content/features/X_train.npy')
y_train = np.load('/content/features/y_train.npy')
print(f'Loaded: X_train {X_train.shape}, y_train {y_train.shape}\n')

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
results = {}

# ---- 1. NCM baseline (no tuning) ----
print('[1/5] Nearest-Class-Mean baseline...')
t0 = time.time()
ncm = NearestCentroid()
s = cross_val_score(ncm, X_train, y_train, cv=cv, n_jobs=-1).mean()
ncm.fit(X_train, y_train)
results['NCM'] = {'best_params': {}, 'best_cv_score': s, 'best_estimator':
ncm}
print(f'  CV accuracy: {s:.4f}   ({time.time()-t0:.1f}s)\n')

# ---- 2. k-NN ----
print('[2/5] k-NN tuning over k in {1,3,5,7,9}...')
t0 = time.time()
knn = GridSearchCV(KNeighborsClassifier(metric='euclidean'),
                   param_grid={'n_neighbors': [1, 3, 5, 7, 9]},
                   cv=cv, scoring='accuracy', n_jobs=-1)
knn.fit(X_train, y_train)
results['kNN'] = {'best_params': knn.best_params_,
                  'best_cv_score': knn.best_score_,
                  'best_estimator': knn.best_estimator_}
print(f'  Best k = {knn.best_params_["n_neighbors"]}, '
      f'CV = {knn.best_score_:.4f}   ({time.time()-t0:.1f}s)\n')

# ---- 3. Linear SVM (smaller C grid based on prior knowledge) ----
print('[3/5] Linear SVM tuning over C in {0.01, 0.1, 1}...')
t0 = time.time()
lin = GridSearchCV(
    Pipeline([('scaler', StandardScaler()),
              ('svm', LinearSVC(max_iter=5000, dual='auto'))]),
    param_grid={'svm__C': [0.01, 0.1, 1]},     # narrowed: C=0.1 won last
time, focus there
    cv=cv, scoring='accuracy', n_jobs=-1)
lin.fit(X_train, y_train)
results['LinearSVM'] = {'best_params': lin.best_params_,
                        'best_cv_score': lin.best_score_,
                        'best_estimator': lin.best_estimator_}
print(f'  Best C = {lin.best_params_["svm__C"]}, '
      f'CV = {lin.best_score_:.4f}   ({time.time()-t0:.1f}s)\n')
```

```
# ---- 4. RBF SVM (with PCA from the start, refined grid) ----
print('[4/5] RBF SVM tuning (PCA-256, refined grid)...')
t0 = time.time()
rbf = GridSearchCV(
    Pipeline([('scaler', StandardScaler()),
              ('pca',    PCA(n_components=256, random_state=42)),
              ('svm',    SVC(kernel='rbf',
decision_function_shape='ovr'))]),
    param_grid={'svm__C':     [1, 10, 100],
                'svm__gamma': ['scale', 0.0005, 0.001]},
    cv=cv, scoring='accuracy', n_jobs=-1)
rbf.fit(X_train, y_train)
results['RBFSVM'] = {'best_params': rbf.best_params_,
                     'best_cv_score': rbf.best_score_,
                     'best_estimator': rbf.best_estimator_}
print(f'  Best (C, gamma) = ({rbf.best_params_["svm__C"]}, '
      f'{rbf.best_params_["svm__gamma"]}), '
      f'CV = {rbf.best_score_:.4f}   ({time.time()-t0:.1f}s)\n')

# ---- 5. sklearn MLP (with PCA, early stopping) ----
print('[5/5] sklearn MLP tuning (PCA-256, early stopping)...')
t0 = time.time()
mlp = GridSearchCV(
    Pipeline([('scaler', StandardScaler()),
              ('pca',    PCA(n_components=256, random_state=42)),
              ('mlp',    MLPClassifier(activation='relu', solver='adam',
                                       max_iter=200, early_stopping=True,
                                       validation_fraction=0.1,
                                       n_iter_no_change=10,
random_state=42))]),
    param_grid={'mlp__hidden_layer_sizes': [(64,), (128,)]},
    cv=cv, scoring='accuracy', n_jobs=-1)
mlp.fit(X_train, y_train)
results['MLP_sklearn'] = {'best_params': mlp.best_params_,
                          'best_cv_score': mlp.best_score_,
                          'best_estimator': mlp.best_estimator_}
print(f'  Best hidden = {mlp.best_params_["mlp__hidden_layer_sizes"]}, '
      f'CV = {mlp.best_score_:.4f}   ({time.time()-t0:.1f}s)\n')

# ---- Summary ----
print('='*60)
print('5-fold CV summary (all classifiers)')
print('='*60)
print(f'{"Classifier":<14}{"CV accuracy":>14}   Best params')
```

```
print('-'*60)
for name, r in sorted(results.items(), key=lambda kv:
-kv[1]['best_cv_score']):
    print(f'{name:<14}{r["best_cv_score"]:>14.4f}   {r["best_params"]}')

# Save
for d in ['/content/features',
          '/content/drive/MyDrive/SIP_Project/features']:
    with open(os.path.join(d, 'cv_results.pkl'), 'wb') as f:
        pickle.dump(results, f)

best = max(results.items(), key=lambda kv: kv[1]['best_cv_score'])
print(f'\nBest by CV: {best[0]} ({best[1]["best_cv_score"]:.4f})')
```

Cell 7-RF:

```
# ============================================================
# Cell 7-RF — Random Forest on updated feature vector (LBP included)
# ============================================================
import os, time, pickle, numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.metrics import accuracy_score, classification_report

X_train = np.load('/content/features/X_train.npy')   # now (1407, 3610)
y_train = np.load('/content/features/y_train.npy')
X_test  = np.load('/content/features/X_test.npy')
y_test  = np.load('/content/features/y_test.npy')

print(f'Feature dims: {X_train.shape[1]}  (includes LBP)')

CLASSES = ['american_football', 'baseball', 'basketball',
           'football', 'golf_ball', 'tennis_ball']

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# ---- 1. Random Forest (no scaling needed — RF is scale-invariant) ----
print('\n[1/2] Random Forest (n_estimators=300)...')
t0 = time.time()
rf = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)
rf_cv = cross_val_score(rf, X_train, y_train, cv=cv,
                        scoring='accuracy', n_jobs=-1).mean()
print(f'  5-fold CV accuracy: {rf_cv:.4f}   ({time.time()-t0:.1f}s)')

# Fit on full train, evaluate on test
```

```
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
rf_test   = accuracy_score(y_test, y_pred_rf)
print(f'  Test accuracy:      {rf_test:.4f}')
print(f'\nClassification report (RF):')
print(classification_report(y_test, y_pred_rf, target_names=CLASSES,
digits=4))

# ---- 2. Linear SVM on new vector (quick sanity check) ----
print('\n[2/2] Linear SVM (C=0.01) on new vector...')
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline

t0 = time.time()
lin = Pipeline([('scaler', __import__('sklearn.preprocessing',
fromlist=['StandardScaler']).StandardScaler()),
                ('svm', LinearSVC(C=0.01, max_iter=5000, dual='auto'))])
lin_cv = cross_val_score(lin, X_train, y_train, cv=cv,
                         scoring='accuracy', n_jobs=-1).mean()
lin.fit(X_train, y_train)
y_pred_lin = lin.predict(X_test)
lin_test   = accuracy_score(y_test, y_pred_lin)
print(f'  CV: {lin_cv:.4f}   Test: {lin_test:.4f}
({time.time()-t0:.1f}s)')

# ---- Summary ----
print('\n' + '='*60)
print('Results with LBP added (new 3,610-dim feature vector)')
print('='*60)
print(f'  Random Forest    CV={rf_cv:.4f}   Test={rf_test:.4f}')
print(f'  Linear SVM       CV={lin_cv:.4f}   Test={lin_test:.4f}')
print(f'  Previous best    CV=0.7612   Test=0.8581  (no LBP)')


# Save RF into results for Cell 8
with open('/content/features/cv_results.pkl', 'rb') as f:
    results = pickle.load(f)

results['RF_LBP'] = {
    'best_params': {'n_estimators': 300},
    'best_cv_score': rf_cv,
    'best_estimator': rf,
    'test_accuracy': rf_test,
    'y_pred': y_pred_rf,
}
```

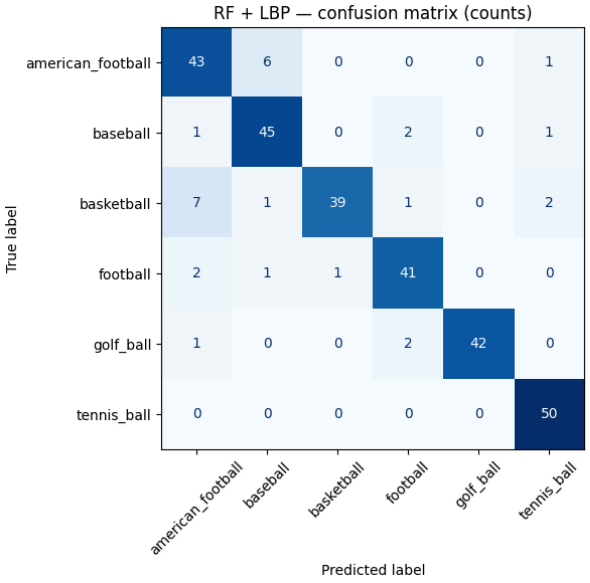
```
for d in ['/content/features',
          '/content/drive/MyDrive/SIP_Project/features']:
    with open(os.path.join(d, 'cv_results.pkl'), 'wb') as f:
        pickle.dump(results, f)
print('\nSaved RF_LBP to cv_results.pkl')
```

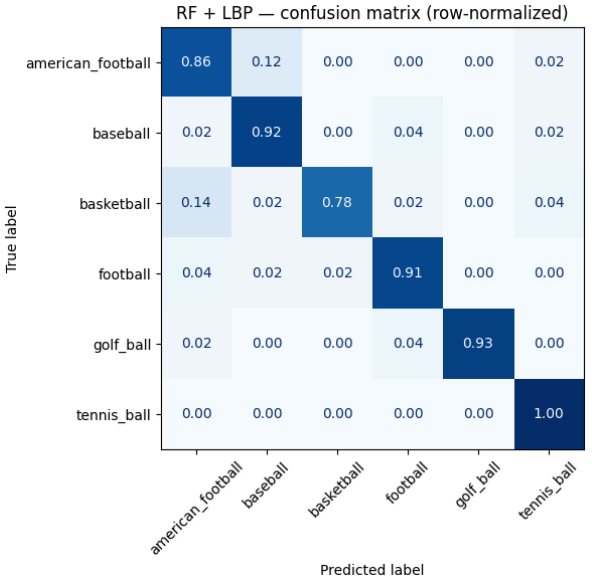
Cell 8:

```
# ============================================================
# Cell 8 — Stage 5: Test-set evaluation (RF + LBP — final result)
# ============================================================
import os, pickle, numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

X_test = np.load('/content/features/X_test.npy')
y_test = np.load('/content/features/y_test.npy')

with open('/content/features/cv_results.pkl', 'rb') as f:
    results = pickle.load(f)

CLASSES = ['american_football', 'baseball', 'basketball',
           'football', 'golf_ball', 'tennis_ball']

print(f'Test set: {X_test.shape[0]} samples, {X_test.shape[1]} dims\n')

r        = results['RF_LBP']
y_pred   = r['y_pred']
best_acc = r['test_accuracy']

print(f'{"="*60}')
print(f'Best classifier: Random Forest + LBP')
print(f'Test accuracy  : {best_acc:.4f}  ({best_acc*100:.2f}%)')
print(f'CV accuracy    : {r["best_cv_score"]:.4f}')
print(f'{"="*60}')

# ---- Progression table (all previous results hardcoded) ----
print(f'\n{"Classifier":<22}{"Test acc":>10}  {"Feature vector":>22}')
print('-'*58)
progression = [
    ('NCM',              0.5121, '3,584-dim (no LBP)'),
    ('k-NN',             0.7266, '3,584-dim (no LBP)'),
    ('MLP (sklearn)',    0.7855, '3,584-dim (no LBP)'),
    ('MLP (PyTorch)',    0.7993, '3,584-dim (no LBP)'),
    ('Linear SVM',       0.8581, '3,584-dim (no LBP)'),
```

```
    ('Linear SVM + LBP', 0.8754, '3,610-dim (with LBP)'),
    ('RF + LBP',         best_acc,'3,610-dim (with LBP)'),
]
for name, acc, note in progression:
    star = ' ★' if name == 'RF + LBP' else ''
    print(f'{name:<22}{acc:>10.4f}  {note:>22}{star}')

# ---- Per-class report ----
print(f'\nPer-class precision / recall / F1 (RF + LBP):')
print(classification_report(y_test, y_pred, target_names=CLASSES, digits=4))

# ---- Confusion matrix ----
cm      = confusion_matrix(y_test, y_pred)
cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
ConfusionMatrixDisplay(cm, display_labels=CLASSES).plot(
    ax=axes[0], cmap='Blues', colorbar=False, values_format='d')
axes[0].set_title('RF + LBP — confusion matrix (counts)')
axes[0].tick_params(axis='x', rotation=45)
ConfusionMatrixDisplay(cm_norm, display_labels=CLASSES).plot(
    ax=axes[1], cmap='Blues', colorbar=False, values_format='.2f')
axes[1].set_title('RF + LBP — confusion matrix (row-normalized)')
axes[1].tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.savefig('/content/features/confusion.png', dpi=120, bbox_inches='tight')
plt.show()

# ---- Top confusion pairs ----
print('Top confusion pairs:')
pairs = [(cm[i,j], CLASSES[i], CLASSES[j])
         for i in range(len(CLASSES)) for j in range(len(CLASSES))
         if i != j and cm[i,j] > 0]
pairs.sort(reverse=True)
for n, t, p in pairs[:8]:
    print(f'  {n:>3}  {t:<20} -> {p}')

# ---- Save ----
for d in ['/content/features',
          '/content/drive/MyDrive/SIP_Project/features']:
    with open(os.path.join(d, 'test_results.pkl'), 'wb') as f:
        pickle.dump({'best_name': 'RF_LBP',
                     'best_acc':   best_acc,
                     'confusion_matrix': cm,
                     'y_pred': y_pred,
```

```
                    'classes': CLASSES}, f)
print('\nSaved test_results.pkl. Stage 5 complete.')
```





Cell 9 :

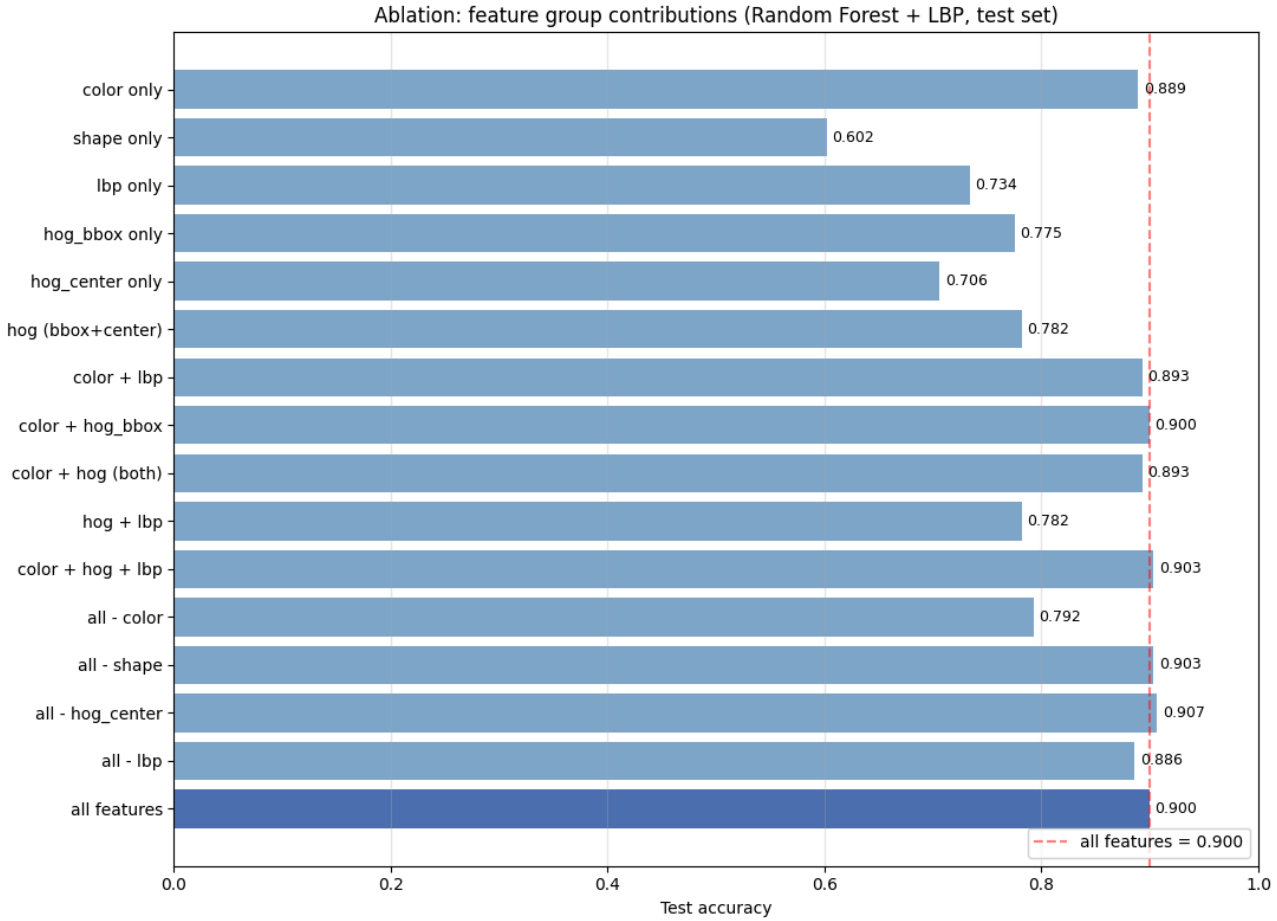
```
# ============================================================
# Cell 9 — Stage 6: Ablation + learning curve (RF + LBP)
# ============================================================
import os, pickle, numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import StratifiedShuffleSplit

X_train = np.load('/content/features/X_train.npy')  # (1407, 3610)
y_train = np.load('/content/features/y_train.npy')
X_test  = np.load('/content/features/X_test.npy')   # ( 289, 3610)
y_test  = np.load('/content/features/y_test.npy')

CLASSES = ['american_football', 'baseball', 'basketball',
           'football', 'golf_ball', 'tennis_ball']

# Feature index ranges — now includes LBP
F = {
    'color':      slice(0, 48),
    'hog_bbox':   slice(48, 1812),
    'hog_center': slice(1812, 3576),
    'shape':      slice(3576, 3584),
    'lbp':        slice(3584, 3610),
}
```

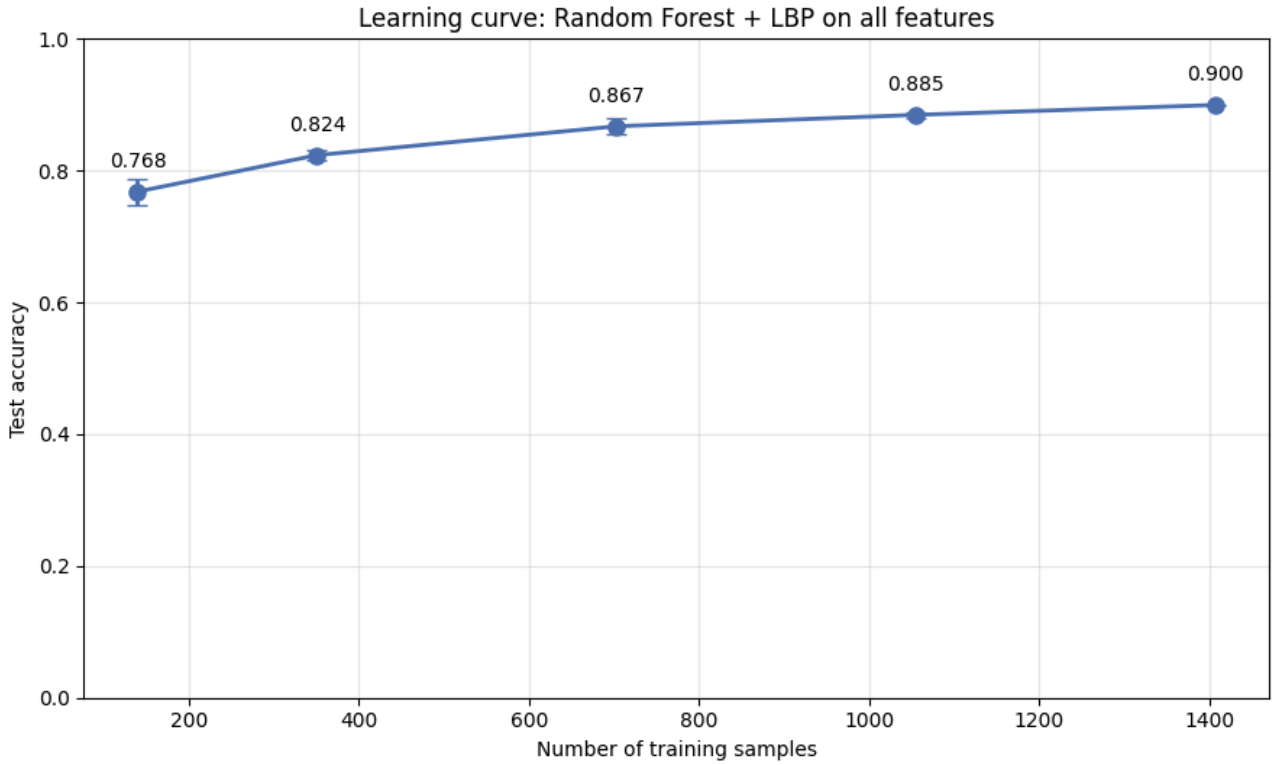
```
def make_pipe():
    return RandomForestClassifier(n_estimators=300, random_state=42,
n_jobs=-1)

def train_and_eval(X_tr, y_tr, X_te, y_te):
    m = make_pipe()
    m.fit(X_tr, y_tr)
    return accuracy_score(y_te, m.predict(X_te))

# ---- ABLATION ----
print('='*60)
print('ABLATION STUDY (Random Forest, 300 trees, with LBP)')
print('='*60)

configs = [
    ('color only',            ['color']),
    ('shape only',            ['shape']),
    ('lbp only',              ['lbp']),
    ('hog_bbox only',         ['hog_bbox']),
    ('hog_center only',       ['hog_center']),
    ('hog (bbox+center)',     ['hog_bbox', 'hog_center']),
    ('color + lbp',           ['color', 'lbp']),
    ('color + hog_bbox',      ['color', 'hog_bbox']),
    ('color + hog (both)',    ['color', 'hog_bbox', 'hog_center']),
    ('hog + lbp',             ['hog_bbox', 'hog_center', 'lbp']),
    ('color + hog + lbp',     ['color', 'hog_bbox', 'hog_center', 'lbp']),
    ('all - color',           ['hog_bbox', 'hog_center', 'shape', 'lbp']),
    ('all - shape',           ['color', 'hog_bbox', 'hog_center', 'lbp']),
    ('all - hog_center',      ['color', 'hog_bbox', 'shape', 'lbp']),
    ('all - lbp',             ['color', 'hog_bbox', 'hog_center', 'shape']),
    ('all features',          ['color', 'hog_bbox', 'hog_center', 'shape',
'lbp']),
]

ablation_results = []
print(f'\n{"Configuration":<26}{"# dims":>8}{"Test acc":>12}')
print('-'*50)
for name, groups in configs:
    cols = np.concatenate([np.arange(F[g].start, F[g].stop)
                           for g in groups])
    Xtr = X_train[:, cols]
    Xte = X_test[:,  cols]
    # RF is scale-invariant — no L2 normalisation needed
    acc = train_and_eval(Xtr, y_train, Xte, y_test)
    ablation_results.append((name, len(cols), acc))
```

```
    print(f'{name:<26}{len(cols):>8}{acc:>12.4f}')

# Plot ablation
fig, ax = plt.subplots(figsize=(11, 8))
labels = [r[0] for r in ablation_results]
accs   = [r[2] for r in ablation_results]
ypos   = np.arange(len(labels))
colors = ['#4c72b0' if 'all features' in l else '#7fa6c9' for l in labels]
ax.barh(ypos, accs, color=colors)
ax.set_yticks(ypos)
ax.set_yticklabels(labels)
ax.set_xlabel('Test accuracy')
ax.set_xlim(0, 1.0)
ax.axvline(accs[-1], color='red', linestyle='--', alpha=0.5,
           label=f'all features = {accs[-1]:.3f}')
for i, a in enumerate(accs):
    ax.text(a + 0.005, i, f'{a:.3f}', va='center', fontsize=9)
ax.set_title('Ablation: feature group contributions (Random Forest + LBP,
test set)')
ax.legend(loc='lower right')
ax.grid(axis='x', alpha=0.3)
ax.invert_yaxis()
plt.tight_layout()
plt.savefig('/content/features/ablation.png', dpi=120, bbox_inches='tight')
plt.show()

# ---- LEARNING CURVE ----
print('\n' + '='*60)
print('LEARNING CURVE (Random Forest, all features + LBP)')
print('='*60)

fractions = [0.10, 0.25, 0.50, 0.75, 1.00]
n_repeats = 3
learning_curve_results = []

print(f'\n{"Train fraction":<18}{"# train":>10}{"Test acc (mean ±
std)":>26}')
print('-'*54)
for frac in fractions:
    accs = []
    for rep in range(n_repeats):
        if frac < 1.0:
            sss = StratifiedShuffleSplit(n_splits=1, train_size=frac,
                                         random_state=42 + rep)
            idx, _ = next(sss.split(X_train, y_train))
```

```
            Xtr, ytr = X_train[idx], y_train[idx]
        else:
            Xtr, ytr = X_train, y_train
        accs.append(train_and_eval(Xtr, ytr, X_test, y_test))
    learning_curve_results.append(
        (frac, len(Xtr), np.mean(accs), np.std(accs)))
    print(f'{f"{int(frac*100)}%":<18}{len(Xtr):>10}'
          f'{f"{np.mean(accs):.4f} ± {np.std(accs):.4f}":>26}')

# Plot learning curve
fig, ax = plt.subplots(figsize=(9, 5.5))
xs = [r[1] for r in learning_curve_results]
ys = [r[2] for r in learning_curve_results]
es = [r[3] for r in learning_curve_results]
ax.errorbar(xs, ys, yerr=es, marker='o', linewidth=2,
            markersize=8, capsize=5, color='#4c72b0')
ax.set_xlabel('Number of training samples')
ax.set_ylabel('Test accuracy')
ax.set_title('Learning curve: Random Forest + LBP on all features')
ax.set_ylim(0, 1.0)
ax.grid(alpha=0.3)
for x, y in zip(xs, ys):
    ax.annotate(f'{y:.3f}', (x, y), textcoords='offset points',
                xytext=(0, 12), ha='center', fontsize=10)
plt.tight_layout()
plt.savefig('/content/features/learning_curve.png', dpi=120,
bbox_inches='tight')
plt.show()

# Save results
analysis = {'ablation': ablation_results,
            'learning_curve': learning_curve_results}
for d in ['/content/features',
          '/content/drive/MyDrive/SIP_Project/features']:
    with open(os.path.join(d, 'analysis_results.pkl'), 'wb') as f:
        pickle.dump(analysis, f)
print('\nSaved analysis_results.pkl, ablation.png, learning_curve.png')

# Quick interpretation
print('\n--- Quick interpretation ---')
top_single = max([r for r in ablation_results
                  if r[0].endswith('only')], key=lambda r: r[2])
print(f'Strongest single feature group : {top_single[0]}
({top_single[2]:.4f})')
all_acc = ablation_results[-1][2]
```

```
print(f'All features                    : {all_acc:.4f}')
no_lbp = next(r for r in ablation_results if r[0] == 'all - lbp')
print(f'All minus LBP                   : {no_lbp[2]:.4f}  '
      f'(LBP contribution: {all_acc - no_lbp[2]:+.4f})')
no_color = next(r for r in ablation_results if r[0] == 'all - color')
print(f'All minus color                 : {no_color[2]:.4f}  '
      f'(color contribution: {all_acc - no_color[2]:+.4f})')
```





10. References

[1] M. Mekhail, Signal and Image Processing Lectures, German International University, Cairo, Egypt, 2025.

[2] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), San Diego, CA, USA, 2005, pp. 886–893.

[3] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Syst., Man, Cybern., vol. 9, no. 1, pp. 62–66, 1979.

[4] C. Rother, V. Kolmogorov, and A. Blake, "GrabCut — Interactive Foreground Extraction using Iterated Graph Cuts," ACM Trans. Graphics, vol. 23, no. 3, pp. 309–314, 2004.

[5] M. K. Hu, "Visual pattern recognition by moment invariants," IRE Trans. Inf. Theory, vol. 8, no. 2, pp. 179–187, 1962.

[6] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273–297, 1995.

[7] T. Ojala, M. Pietikainen, and T. Maenpaa, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," IEEE Trans. Pattern Anal. Mach. Intell., vol. 24, no. 7, pp. 971–987, 2002.

[8] L. Breiman, "Random Forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[9] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[10] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019, pp. 8024–8035.

[11] OpenCV, "OpenCV Documentation," [Online]. Available: https://docs.opencv.org. [Accessed: May 7, 2026].